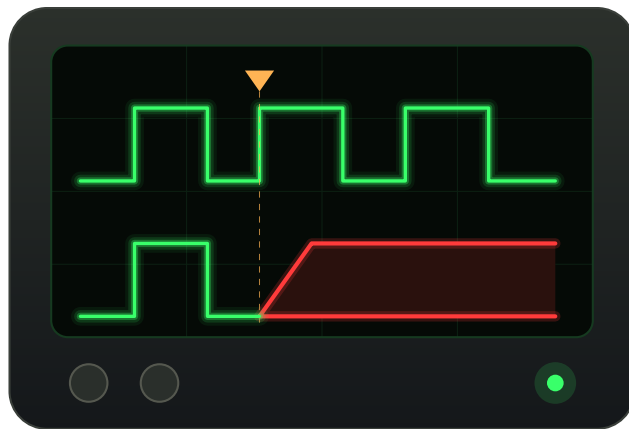




LibreILA

LibreILA User Manual

Yash Paritkar ([github/yashparitkar](https://github.com/yashparitkar))

August 2026

Document History

Version	Date	Comments
0.0	25 July 2026	Document created.

Contents

Document history	2
1 Introduction	11
1.1 What LibreILA is	11
1.2 Using LibreILA	11
1.3 Document Map	12
1.4 Conventions	12
2 Getting Started	15
2.1 Prerequisites	15
2.1.1 Installing the host packages	16
2.1.2 Creating a Python virtual environment	16
2.2 Getting the sources	17
2.2.1 Cloning the repository	17
2.2.2 Importing the Libero project	17
2.2.3 Importing the SoftConsole project	19
2.3 Your first capture	21
2.4 Conclusion	26
3 The Code Generator	27
3.1 Working of the code generator	27
3.2 Writing configuration.csv	28
3.3 Writing portmap.csv	29
3.3.1 Choosing the signal type	30
3.3.2 Using bidirectional signals	30
3.4 Running the generator	31
3.5 Building a generic logic analyzer	32
3.6 Probing multiple channels in same build	32
4 Integrating LibreILA into a Design	33
4.1 Files to add	33
4.2 Core or wrapper	34
4.3 Splicing the probe into a link	34
4.4 Clocks and reset	35
4.5 Timing constraints	35
4.6 Connecting the UART	35
4.7 Checking the build	36
5 The Software Stack	37
5.1 Architecture	37
5.2 The session	37
5.3 The register driver	38
5.4 The trigger translator	38
5.5 The VCD writer	39

6	The Command Line Interface	41
6.1	Telling the tool about a core	41
6.2	The verbs and the order they run in	41
6.3	Configuring the trigger	42
6.4	Worked scenarios	43
6.5	Debugging the link	43
6.6	Exit statuses	44
7	The Graphical Interface	45
7.1	Starting it	45
7.2	Tabs and connections	45
7.3	Running a capture	45
7.4	What it does not do yet	51
8	The Baremetal C Driver	53
8.1	Adding the driver to a project	53
8.2	Definitions you must supply	53
8.3	Initialising	54
8.4	Configuring the trigger	54
8.5	Running a capture	55
8.6	Several cores from one binary	56
8.7	Porting to another toolchain	56
9	Reading the Capture	59
9.1	The VCD file	59
9.2	The trigger marker	59
9.3	The time axis	60
9.4	In the waveform viewer	60
9.5	Reading the raw buffer yourself	60
10	Troubleshooting	61
10.1	Generation and build	61
10.2	The link	62
10.3	The trigger	62
10.4	The capture	63
A	Simulations	65
A.1	The test suite	65
A.2	What each test proves	66
A.3	Running one on its own	66

List of Figures

2.1	The testing setup.	16
2.2	Exploring the block design.	18
2.3	Run PROGRAM Action.	18
2.4	Importing the project.	19
2.5	Opening the SoftConsole project.	20
2.6	The Debug view.	21
2.7	MSS UART connected.	22
2.8	Configuring PKTGEN.	23
2.9	Launching LibreILA.	23
2.10	Configuring and arming the ILA.	24
2.11	Firing PKTGEN, capture done.	25
2.12	The GTKWave window.	26
4.1	Register to register paths out of the core, stock build.	36
7.1	Two stored cores, both disconnected. The geometry reads – because only the UID is known from the device store.	46
7.2	The first tab connected. The information box and the conditions table both come from the core.	47
7.3	An edited trigger: TDATA matched in hex with the top eight nibbles left as don't cares, TVALID high, the rest ignored, on a rising edge with 512 samples of history. The * on the apply button says the core has not been told yet.	48
7.4	Applied. The status bar shows the flat pair the table became, which is the same pair <code>--set-trigger-condition</code> and <code>--set-trigger-mask</code> take, and the rows now read back as the core holds them.	49
7.5	Armed and waiting, with the timer counting.	50
7.6	Read out and written. The status bar gives the sample count and the row the trigger fired on.	51

List of Tables

1.1	The two deliverables and the chapters each one leads to.	12
1.2	The three documents and what each is for.	12
1.3	Conventions used in this manual.	13
2.1	What each prerequisite is for.	15
3.1	The code generation directives.	27
3.2	The generics that matter in practice. Full ranges are in the datasheet.	28
3.3	The templates supplied with the generator.	30
3.4	The probe signal types.	30
3.5	The generator's flags.	31
4.1	The generated files and what each one is.	33
4.2	The two builds compared. The resource figures are the datasheet's, measured on PolarFire SoC.	34
5.1	The modules of the python driver.	37
5.2	The symbolic reasons an <code>OperationError</code> carries.	38
5.3	The trigger translation functions.	39
6.1	The verbs, and the fixed order they are applied in.	42
6.2	Exit statuses of the command line front end.	44
8.1	The files of the baremetal driver.	53
8.2	The build time definitions, which size arrays and nothing else.	54
10.1	Symptom to section.	61
A.1	The three test groups and what each one needs.	65
A.2	The HDL simulations.	66
A.3	The host side tests.	66

List of Acronyms

AXI	Advanced eXtensible Interface
CDC	Clock Domain Crossing
FIFO	First In First Out (queue)
FPGA	Field Programmable Gate Array
GUI	Graphical User Interface
HDL	Hardware Description Language
ILA	Integrated Logic Analyzer
IP	Intellectual Property
LSRAM	Large Static Random Access Memory
OHL	Open Hardware Licence
RTL	Register Transfer Level
SDC	Synopsys Design Constraints
UART	Universal Asynchronous Receiver Transmitter
VCD	Value Change Dump

Chapter 1

Introduction

This manual is about using LibreILA. It contains a worked example of a capture on hardware, modifying the core, wiring into the design, driving it from either host, reading back what it captured, using the GUI and some basic troubleshooting. Although care is taken to write every minor detail down, it is assumed that the reader is familiar with using FPGA toolchains, the command line, and a serial terminal.

1.1 What LibreILA is

LibreILA is an in-system logic analyzer: a small core spliced into a link inside the FPGA fabric, which watches that link go past and records it. Any net in the RTL can be probed. While armed it writes one sample of the probed signals into a circular buffer in fabric RAM on every sampling clock edge, so the buffer always holds the most recent window of history. Once a condition specified by the user is met, the core stops writing and the buffer is frozen after specified post-trigger samples. The user can then read the buffer back into a VCD file on the host, and open it in a waveform viewer to see what happened.

The splice is pass-through. The core's two probe ports are shorted combinatorially, so the link it sits in the middle of behaves exactly as it did before and pays no latency for being watched. The two mirrored ports allow easy connection when using GUI based design tools.

1.2 Using LibreILA

The core is delivered two ways, and which one you build decides which host drives it and which chapters of this manual you need.

`libre_ila` is the core itself, with an AXI4Lite slave port. It suits a design that already carries a bus master: a hard or soft processor, or anything else that can write registers. The baremetal C driver under `drivers/baremetal/` drives it from firmware, so a capture never leaves the chip and no PC has to be attached to take one. Chapter 8 covers that path on its own, end to end.

`libre_ila_uart` wraps the same core with a UART and a packet parser, putting the register map at the end of two wires instead of on a bus. Nothing inside the core changes. It suits a design with no bus master to spare, or a board you would rather drive from a laptop. The python driver under `drivers/python/` drives it from a PC, either from the command line (Chapter 6) or from a window (Chapter 7). This makes the usage very similar to a commercial logic analyzer.

Everything on either side of that choice is shared. Both builds come out of the same generator and are wired into a design the same way, and both capture into the same buffer and are read back into the same VCD. Table 1.1 is the short version.

Build	Driven by	Chapters
<code>libre_ila</code>	the C driver, from firmware	3 , 4 , 8 and 9
<code>libre_ila_uart</code>	the python driver, from a PC	3 , 4 , 6 or 7 , and 9

Table 1.1: The two deliverables and the chapters each one leads to.

The remaining chapters serve both. Chapter [5](#) describes how the python driver is put together, for a reader scripting against it rather than using the front ends. Chapter [10](#) is the symptom-to-fix list, and Appendix [A](#) runs the test suite with no hardware attached. If you are here for the first time, start with Chapter [2](#) whichever build you are eventually after: it takes a capture on a stock build with the wiring already done, so the pieces have names before you have to choose between them.

Interestingly enough, the `libre_ila_uart` build is simply a `libre_ila` core with a Universal Asynchronous Receiver Transmitter (UART) to AXI4Lite bridge around it. If end user demands more speed, HDMI, USB or Ethernet bridges could be added in the same way.

1.3 Document Map

Three documents describe LibreILA.

Document	Answers	Contents
This manual	how do I use it	Prerequisites and a first capture, the code generator, integrating a core into a design, both drivers, reading the capture, and troubleshooting
The datasheet	what is it and what does it guarantee	Probe and port structure, capture and trigger behaviour, clock domains, reset, timing, the core parameters, the AXI4Lite register map register by register, and the UART packet format
The READMEs	how does it work inside	One per directory, next to the code they describe. Implementation notes for someone changing LibreILA itself

Table 1.2: The three documents and what each is for.

The datasheet is the specification, manual is user guide. Both PDFs live in `docs/`, built from the sources in `docs/tex/`, and `REGISTER_MAP.csv` at the repository root carries the same register map one row per field, for a script that would rather not read a PDF.

1.4 Conventions

Table [1.3](#) lists the conventions this manual holds to throughout.

Written as	Means
<code>make sim</code>	A command, shown without a shell prompt so it can be copied whole. Unless the text says otherwise it is run from the root of your clone
<code>{repo}</code>	The root of your clone, wherever you put it. Every other path is relative to it, as in <code>drivers/python/libre_ila.py</code>
<code>TRIG_CFG</code>	A register or one of its fields, spelt as the datasheet register map spells it
<code>G_SAMP_BUFF_DEPTH</code>	A generic, fixed at synthesis and set in <code>configuration.csv</code>
<code>0b001x1</code>	A trigger pattern, where <code>x</code> is a bit that is not compared
bit 0	The least significant bit, and the first signal listed in <code>portmap.csv</code> . Bit order is counted that way everywhere

Table 1.3: Conventions used in this manual.

Two names are kept apart deliberately. *The C driver* always means the baremetal driver for `libre_ila` and *the python driver* always means the host driver for `libre_ila_uart`; neither is ever called just “the driver”, because the two share a register map and nothing else.

This manual is a work in progress. Anything still to be written is marked **TODO: like this** and prints in red, so a gap is visible rather than silent.

Chapter 2

Getting Started

In this chapter, we will walk through the steps to get the example design up and running on the Microchip PolarFire SoC Discovery Kit. We will cover the prerequisites, how to get the sources, and how to perform your first capture using the LibreILA toolchain.

2.1 Prerequisites

The full toolchain is tested using following:

- FPGA: Microchip PolarFire SoC Discovery Kit (mpfs-disco-kit)
- Libero SoC v2025.2
- SoftConsole v2022.2
- Ubuntu 22.04.3 LTS
- Python 3.10.0
- Packages for driver: pyserial (v3.5)
- Packages for GUI: PySide6 (v6.5.1)
- Rpi Pico with `debugprobe_on_pico` firmware flashed as USB-TTL adapter (other serial adapters will also work)

The tools might or might not work on other versions, but the manual is written for the above. The reader is expected to have a working knowledge of Linux and Python, and to be able to install software and run commands in a terminal.

Not all of it is needed for all of it. Table 2.1 says what each piece buys; leave a row out and everything that does not depend on it still works.

Tool	Needed for
Python 3.10 or later	everything on the host
pyserial	the python driver, both front ends
PySide6	the graphical front end, <code>--gui</code> , only
USB-TTL adapter	reaching <code>libre_ila_uart</code> from the PC
Libero SoC	building a core into a design and programming the FPGA
SoftConsole	the baremetal C driver, and the demo application this chapter runs
GHDL	optional, runs the simulations under <code>tests/hdl/</code>
GTKWave	optional, opens the captured <code>.vcd</code> ; any VCD viewer will do

Table 2.1: What each prerequisite is for.

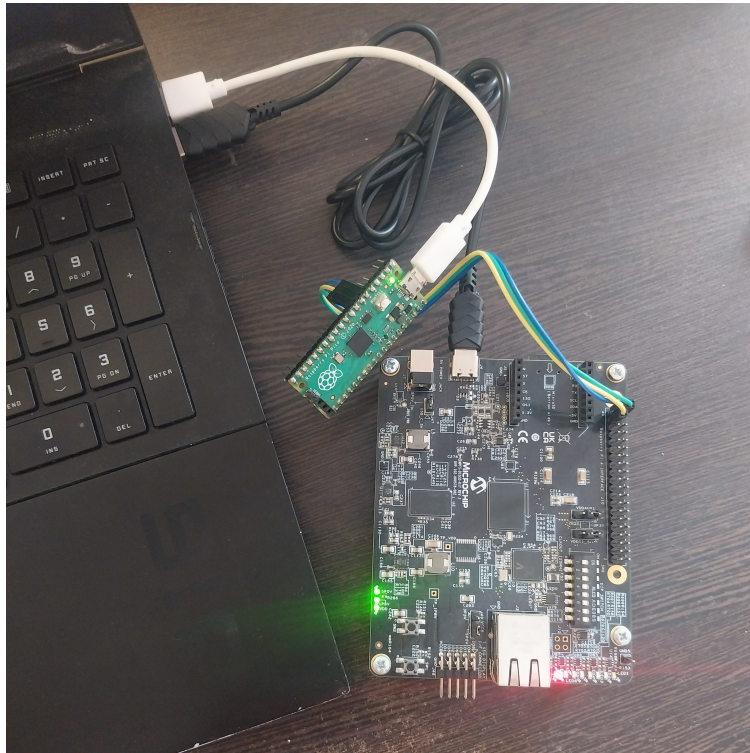


Figure 2.1: The testing setup.

2.1.1 Installing the host packages

Both python packages are in apt on Debian and Ubuntu:

```
sudo apt install python3-serial
sudo apt install python3-pyside6.qtwidgets python3-pyside6.qtgui
```

PySide6 ships one package per Qt module there, so `python3-pyside6` is not a package and `python3-pyside6.qtc core` on its own is not enough. Everything except `--gui` works without either of the two.

Ubuntu 22.04 predates PySide6's apt packaging, so on the tested system those two packages do not exist at all. Install it into a virtual environment instead, as in Section 2.1.2.

The optional tools are one line as well:

```
sudo apt install ghdl gtkwave
```

Neither is needed to build a core or take a capture. GHDL runs the simulations under `tests/hdl/`, and GTKWave is what the driver's `--waveform-viewer` defaults to when it opens a capture for you.

Finally, reading a serial port on Ubuntu needs membership of the `dialout` group. Add yourself once and log out and back in, otherwise every connection fails with a permission error on `/dev/ttyUSB0`:

```
sudo usermod -aG dialout $USER
```

2.1.2 Creating a Python virtual environment

This is how PySide6 is installed on a distribution that does not package it, and it is the tidier route on any distribution, since nothing lands system wide.

1. Install Python 3.10 or later, pip and the venv module:

```
sudo apt install python3 python3-pip python3-venv
```


2. In the root of the repository, create a virtual environment named **venv-libreila**:

```
python3 -m venv venv-libreila
```

3. Activate the virtual environment:

```
source venv-libreila/bin/activate
```

4. Install the required packages:

```
pip install pyserial PySide6
```

The name and the place matter. The root **Makefile** and the GUI test under **tests/python/05_gui/** both look for **venv-libreila** at the repository root and fall back to it when the system **python3** has no **PySide6**, so a virtual environment put anywhere else has to be activated by hand for every command in this manual.

To check that one of the two has it, without running anything else, from the repository root:

```
make check-pyside6
```

It prints an install line if neither the system **python3** nor the virtual environment has **PySide6**, and stays silent when one of them does.

2.2 Getting the sources

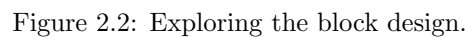
2.2.1 Cloning the repository

Make a local clone of the repository,

```
git clone https://github.com/yashparitkar/LibreILA
```

2.2.2 Importing the Libero project

1. Open Libero
2. In the Project tab → Open Project → Browse to **{repo}/tests/full/libre_ila_uart/libre_ila_uart_libero/libre_ila_uart_libero.prjx**
3. You can view the block design in **Design Heirarchy**, it contains the **libre_ila**, **axi4smaster_pktgen** (to generate dummy axistream traffic), a simple heartbeat ip and other MPFS blocks. Notice how the LibreILA core is connected to the AXI4Stream link between the **pktgen** and the slave is kept open while tying the **TREADY** high to ensure the **pktgen** can always send data.



- Figure 2.3: Run PROGRAM Action.

6. The design uses B14 (RPI Pin 38) as UART TX and E13 (RPI Pin 40) as UART RX. This can be verified in the `libreila.pdc` constraints file.
7. Connect the USB-TTL adapter to the board, and connect the TX of the adapter to B14 and RX to E13. The GND of the adapter should be connected to any GND pin on the board.

2.2.3 Importing the SoftConsole project

1. Open SoftConsole
2. Right click in the Project Explorer tab → Import → General → Existing Projects into Workspace → Next → Browse → Select `{repo}/tests/full/libre_ila_uart/libre_ila_uart_sc` → Finish

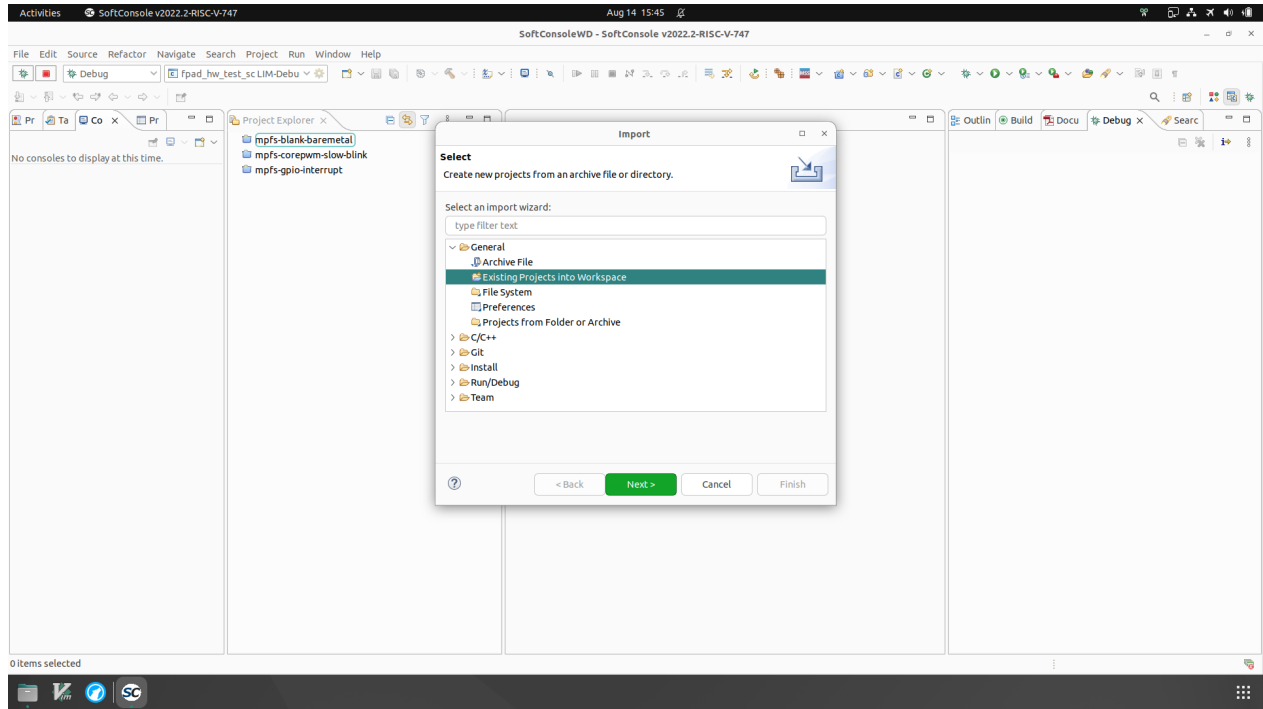


Figure 2.4: Importing the project.

3. You can view the source code in `src` folder, the file `src/application/hart1/u54_1.c` contains the main function which initializes the UART and sends some dummy data over AXI4Stream to the `libre_ila` core.

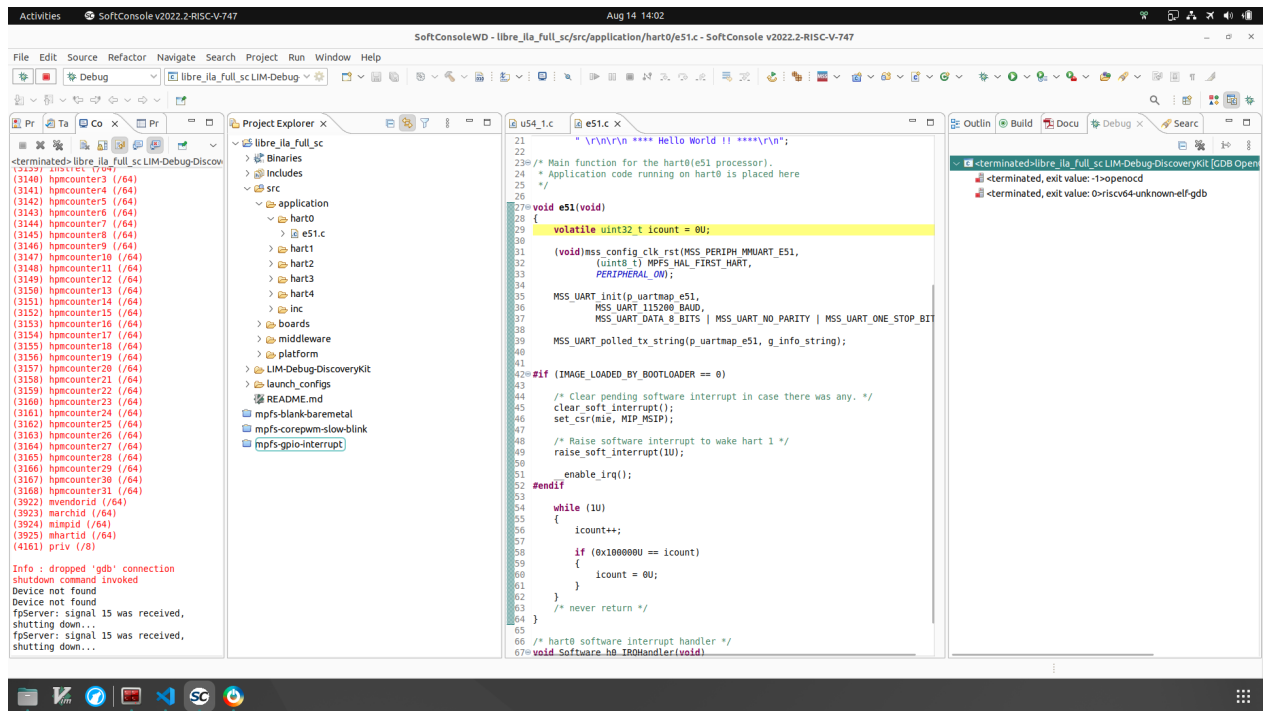


Figure 2.5: Opening the SoftConsole project.

4. Change the Build Configuration to LIM-Debug-DiscoveryKit
5. Build the project by clicking on the hammer icon in the toolbar.
6. Debug the project by clicking on the bug icon in the toolbar, it should program the SoC and start the application.

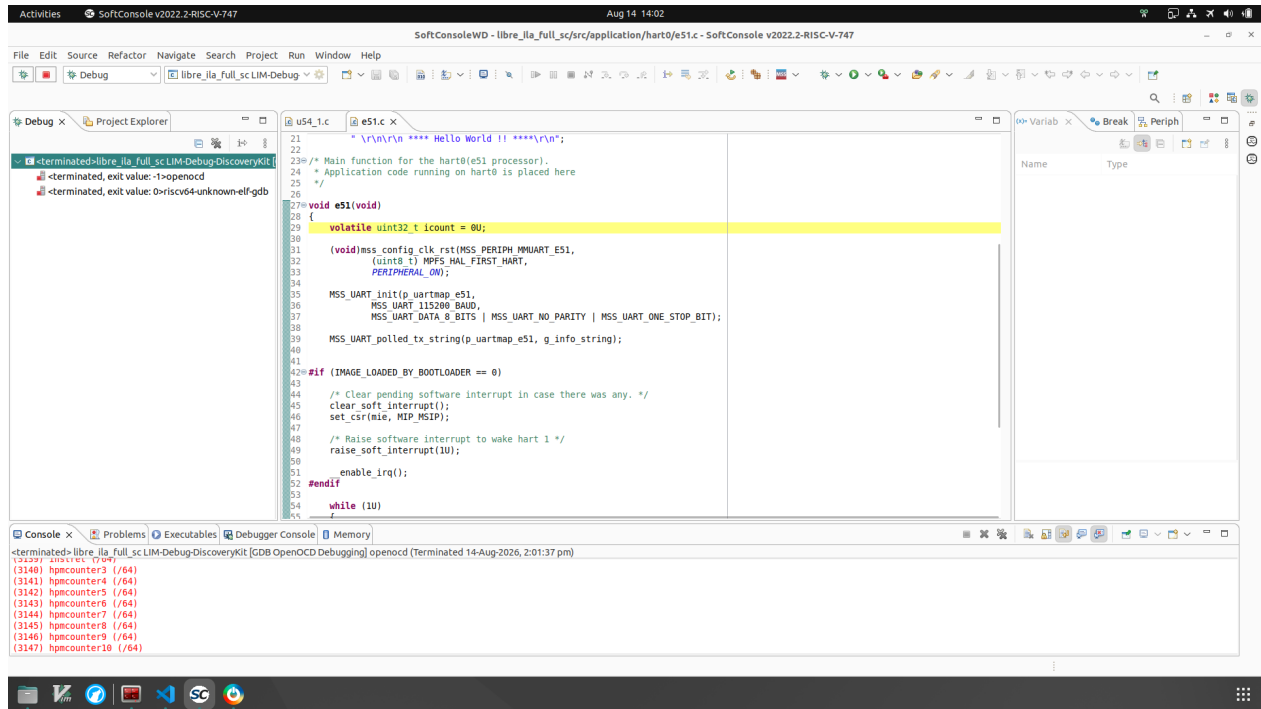


Figure 2.6: The Debug view.

7. Once the DDR training is complete, the application will stop at breakpoint in `src/application/hart0/u54.0.c` file, click on the Resume button in the toolbar to continue the execution of the application.

2.3 Your first capture

1. Start the debug project on SoftConsole
2. Open a the serial terminal to connect with the MSS UART, you can use any serial terminal of your choice, we will use `minicom` here. Note that, if the drivers for the MPFS is installed properly, you will see three serial devices in `/dev/` directory, one of them will be streaming the output from the MSS UART. In our case, it is `/dev/ttyUSB2`.

```
minicom -D /dev/ttyUSB2 -b 115200
```

3. In the serial terminal, you should see the following output:

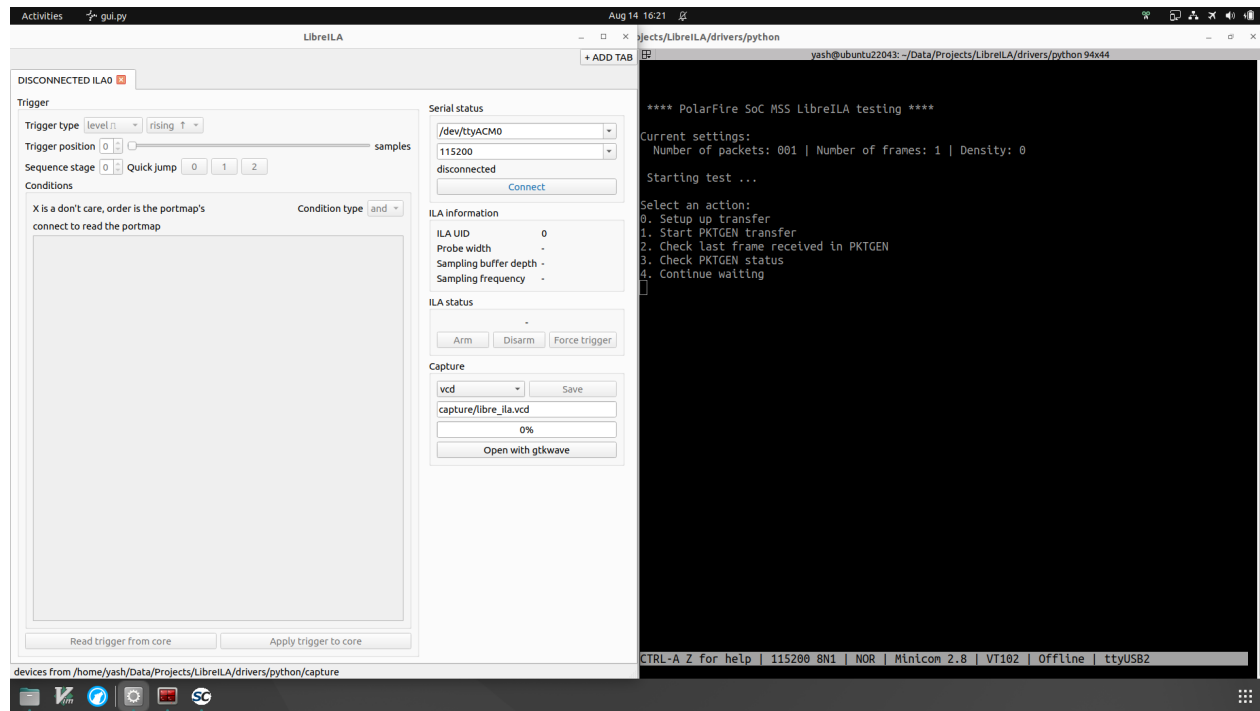


Figure 2.7: MSS UART connected.

4. Configure the pktgen using the menu options, we are using following configuration:

```

Packet size: 250 frames
Packet count: 250 packets
Packet density: bubble per 2 frames

```

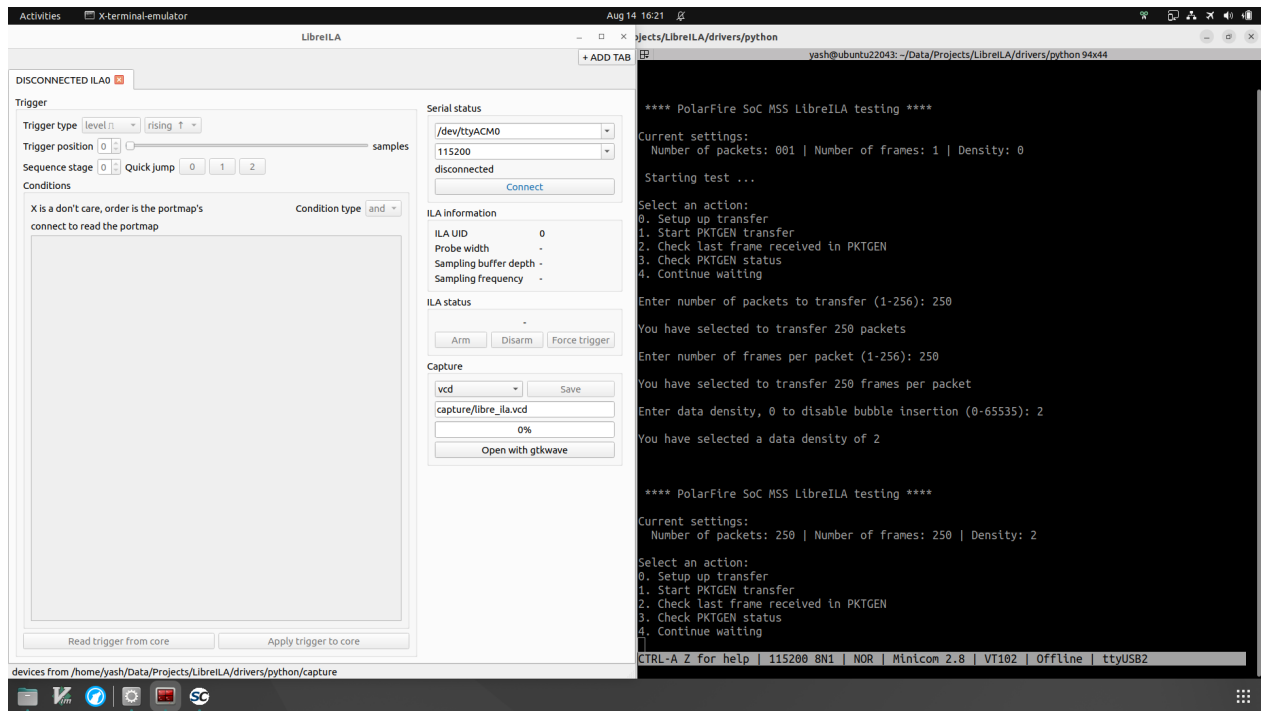


Figure 2.8: Configuring PKTGEN.

5. Open the GUI front end, you can use the following command to open the GUI:

```
python3 libreila.py --gui
```

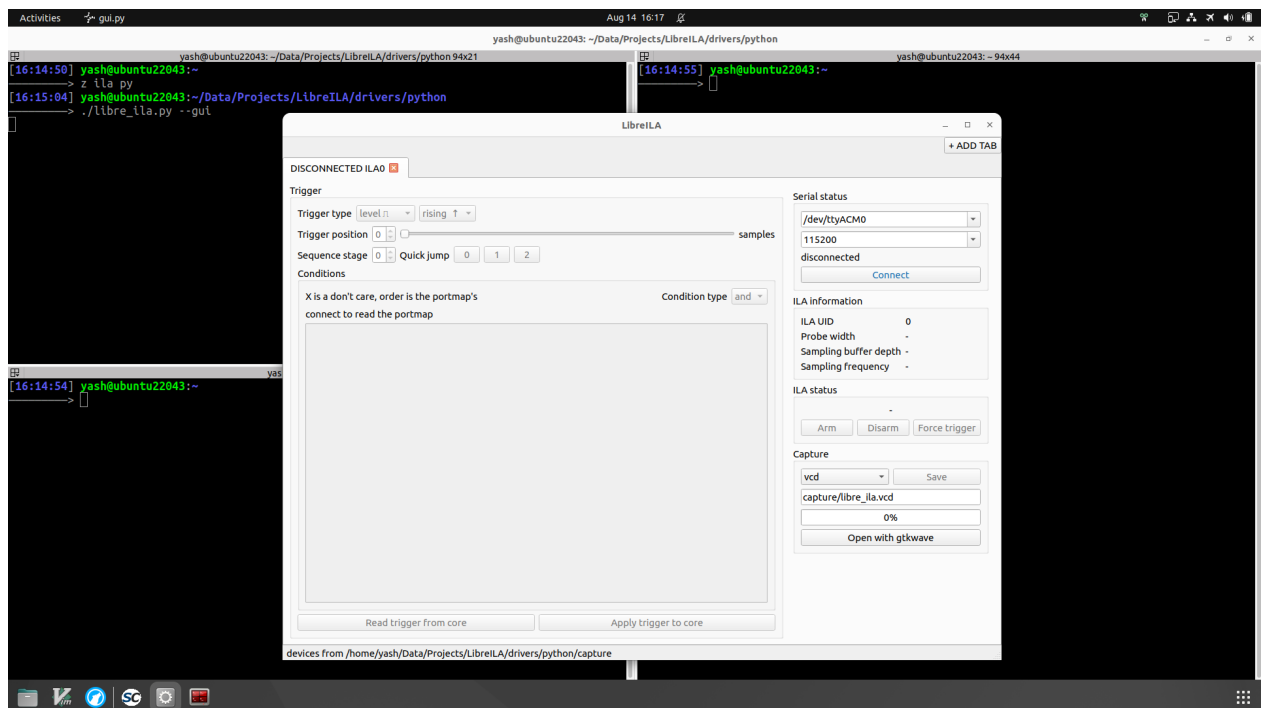


Figure 2.9: Launching LibreILA.

6. In the GUI, click on **Add Device** and select the serial port to which the USB-TTL adapter is connected, in our case it is `/dev/ttyACM0`. Click on **Connect** to connect to the device.
7. Set the trigger edge, rising and move the position to somewhere middle and trigger condition to **TVALID** and **TLAST** rising. Click on **Arm** to arm the trigger.

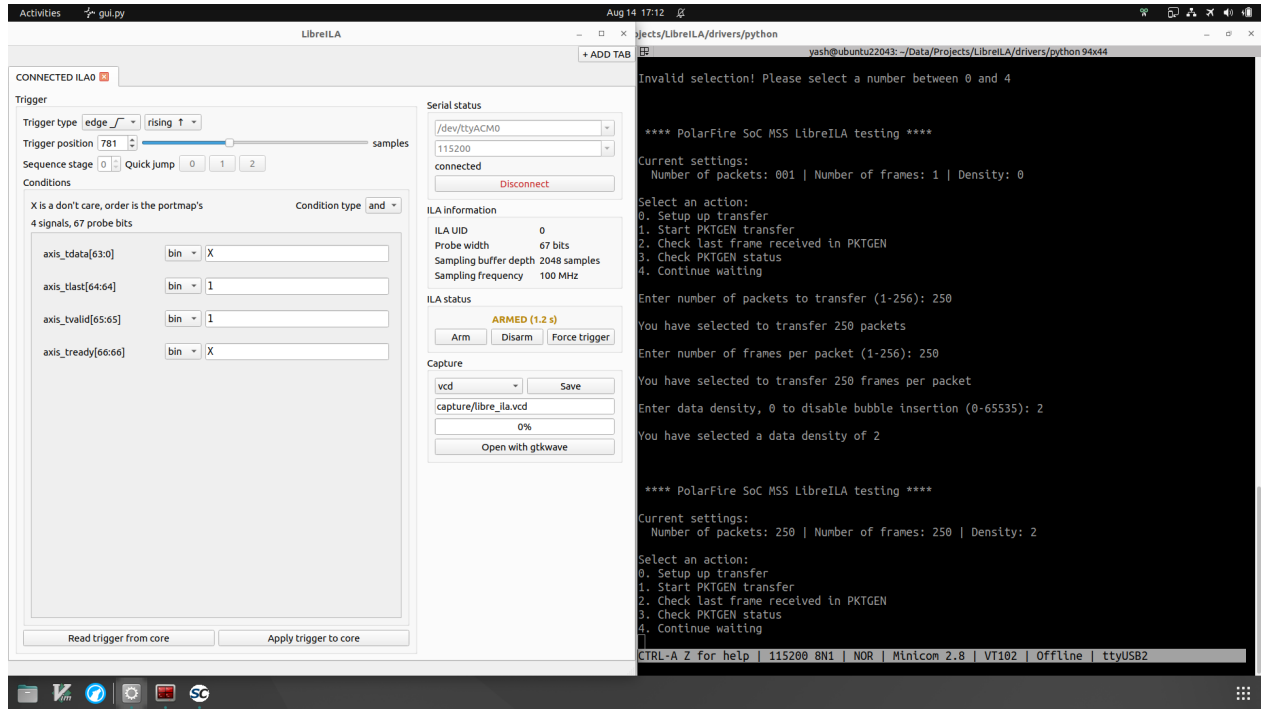


Figure 2.10: Configuring and arming the ILA.

8. Fire the packet generator by selection option 1. **Start PKTGEN transfer** on the MSS serial terminal
9. The ILA should've captured the data and the ILA status will change to **Triggered** and then to **Done** quickly.

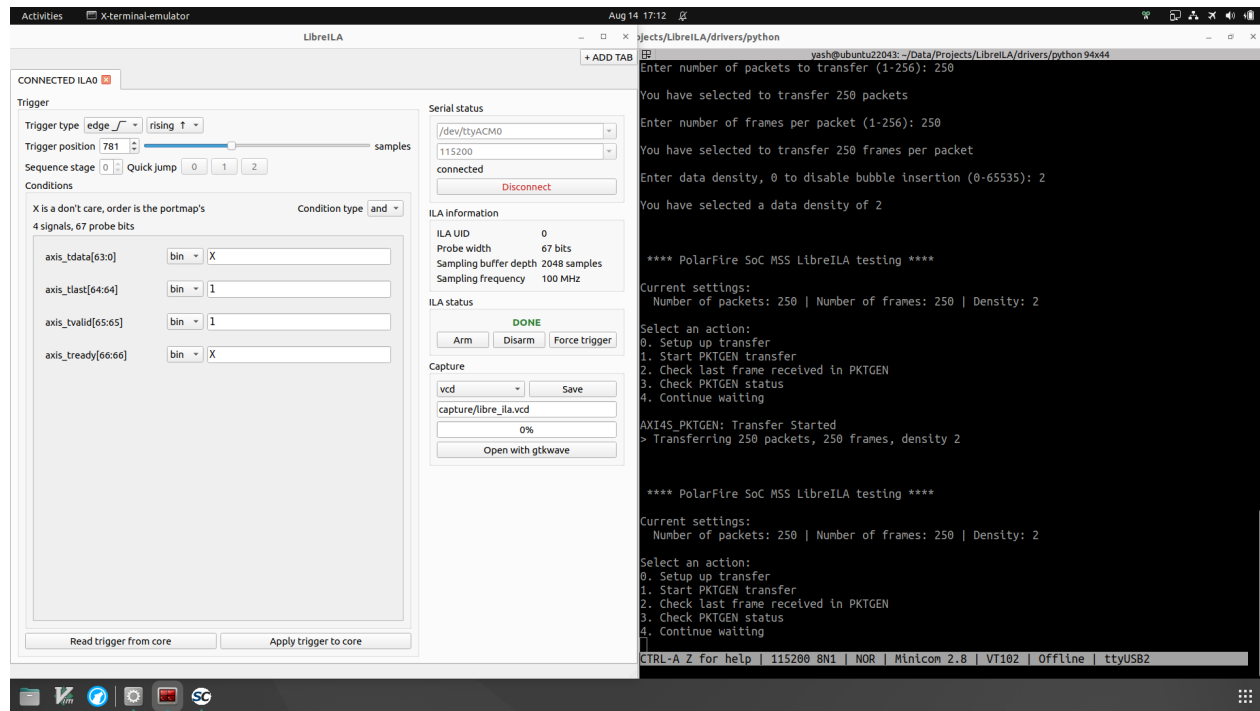


Figure 2.11: Firing PKTGEN, capture done.

10. Press **Save** in the **Capture** section to read data from the ILA and save it to a VCD file.
11. Press **Open with gtkwave** to open the capture in gtkwave
12. In gtkwave, right click on the `libre_ila` → Recure Import → Insert to add signals to waveform window
13. Tip: click on the "Magnifying glass with green tick" to fit the capture in the window

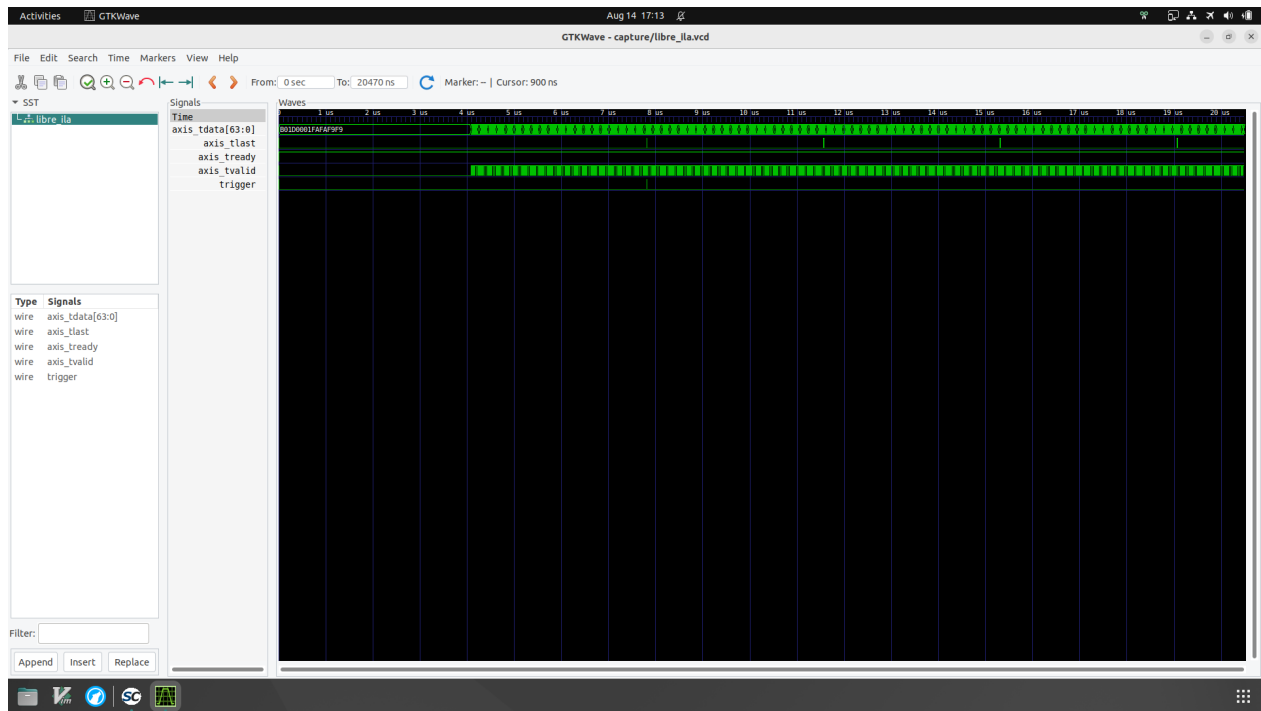


Figure 2.12: The GTKWave window.

2.4 Conclusion

In this chapter, we have walked through the steps to get the example design up and running on the Microchip PolarFire SoC Discovery Kit. We have covered the prerequisites, how to get the sources, an overview of how to connect the LibreILA, and how to perform your first capture using the LibreILA toolchain. You should now be able to replicate the steps and perform your own captures using the LibreILA toolchain.

In further chapters, we will cover how to modify the LibreILA core to capture different signals, how to use the driver to configure the core and how to use the GUI front end to visualize the captured data. Similar steps can be followed to adapt this to your own design, toolchain and board.

Chapter 3

The Code Generator

A build of the core is described entirely by two CSV files in `codegen/`. `configuration.csv` lists the generics and their values, one line per generic, and `portmap.csv` lists the signals to probe and their widths, one line per signal. The generator reads both, finds the VHDL templates in `hdl/`, replaces the directives in them with what the two files describe, and writes a complete set of sources to `gen_axis/` for the stock build or to `gen/` otherwise. Nothing is ever written back into `hdl/`, so the templates there stay the single source.

The portmap outlives the generation step. The python driver reads the same file to name the signals in the VCD and to number the trigger bits, so one file describes the probe for the hardware and for the host at once. What comes out of the generator is ordinary VHDL with no further tooling attached to it; Chapter 4 takes it from there.

3.1 Working of the code generator

The code generator is more of a find and replace tool than a compiler. Each templated file is walked twice: once to rewrite the generic defaults, and once to expand the directives.

The generics pass is the simple one. `configuration.csv` is read, every value is checked against the same limits the HDL asserts at elaboration, and the default of each named generic is rewritten in place. Only declarations that already carry a `:=` are touched, which confines the pass to the entity and leaves the component declarations in the wrapper alone. `G_PROBE_WIDTH` is the one generic that is not read from the configuration at all: it is summed from the portmap, which is why it must be generated rather than set by hand.

The directives pass works on the `^^XX` markers that sit in the comment headers of the templates. The generated block is inserted after the run of comment lines that immediately follows the directive line, so the heading and its closing rule stay above the generated code rather than being buried in it. The six directives are listed in Table 3.1.

Directive	File	Expands to
<code>^^DI</code>	<code>both</code>	The probe port declarations, slave side
<code>^^DO</code>	<code>both</code>	The probe port declarations, master side
<code>^^MI</code>	<code>libre_ila_uart</code>	The slave side associations in the port map of the core instance
<code>^^MO</code>	<code>libre_ila_uart</code>	The master side associations in the port map of the core instance
<code>^^SH</code>	<code>libre_ila</code>	The pass through assignments between the two sides
<code>^^MX</code>	<code>libre_ila</code>	The <code>w_probe</code> concatenation that the sampler writes

Table 3.1: The code generation directives.

A directive may appear more than once in a file. The wrapper carries `^^DI` and `^^DO` twice, once in its own entity and once in the component declaration of the core, and the indentation of each generated block

is taken from the directive line itself, so the same block serves both depths. In the template, `^^DI` is nothing but a header:

```

— Probe Slave ^^DI —
— Faces the master of the probed link , so it carries the port
— directions of that link 's slave .

```

and with the stock portmap it comes out as:

```

— Probe Slave ^^DI —
— Faces the master of the probed link , so it carries the port
— directions of that link 's slave .

probe_slave_axis_tdata   : in      std_logic_vector(63 downto 0);
probe_slave_axis_tlast   : in      std_logic;
probe_slave_axis_tvalid  : in      std_logic;
probe_slave_axis_tready  : out     std_logic;

```

Which files come out is decided by `GEN.TYPE`: 0 emits the bare AXI4Lite core alone, and 1 emits it together with the UART wrapper and the `fifo` and `uart` blocks it instantiates. Those last two carry no directives and are copied verbatim.

In case of any error, the generator aborts the run before writing anything. That includes a directive that is listed for a file but never found in it, which is treated as an error rather than as nothing to do: a silently dropped block would not fail here at all, it would elaborate much later as an unconnected port, and that is a far worse thing to debug than a generator that refuses to run.

3.2 Writing configuration.csv

One line per generic, written as `generic,type,value`, where `type` is the VHDL type of the value and is one of `integer`, `natural` or `string`. Blank lines and lines starting with `#` are ignored. Every generic in Table 3.2 except `G_UID` has to be present, and a missing one aborts the run; `G_UID` may be left out entirely, which is the same as setting it to zero. A `natural` may be written in hex, so `G_UID,natural,0x0BADCODE` is read as 195936478, with a ceiling of `0x7FFFFFFF` because a VHDL `natural` cannot hold anything with the top bit set. An `integer` is read as decimal only.

Generic	Stock	What it decides
<code>GEN.TYPE</code>	1	Which files come out: 0 the core on its own, 1 the core plus the UART wrapper
<code>G.SAMP_BUFF_DEPTH</code>	2048	How many samples the buffer holds, and with the probe width, how much fabric RAM the core costs. A power of two greater than one
<code>G.EXTERNAL_TRIG</code>	0	1 replaces the trigger comparator with the <code>i_ext_trig</code> pin, and the pattern and mask registers are then ignored
<code>G.SAMP_CLK_FREQ</code>	100 000 000	Nothing in the logic. It is reported to the host, which builds the time axis of the VCD from it
<code>G.AXIL_CLK_FREQ</code>	100 000 000	The watchdog period and the UART divider, so it has to match the clock the bus really runs at
<code>G.BAUD_RATE</code>	115 200	Wrapper only. The line rate, 8N1
<code>G.UART_RX_FIFO_DEPTH</code> <code>G.UART_TX_FIFO_DEPTH</code>	1024 1024	Wrapper only. Bytes buffered in each direction. Powers of two
<code>G.UID</code>	0	Nothing in the logic. It is read back at UID so that several cores in one system can be told apart

Table 3.2: The generics that matter in practice. Full ranges are in the datasheet.

The two clock frequencies are worth keeping apart, because only one of them configures anything. Getting `G_SAMP_CLK_FREQ` wrong costs a capture that reads back with the wrong time base and nothing else, and it can be corrected after the fact. Getting `G_AXIL_CLK_FREQ` wrong costs a UART that does not talk, because the divider is derived from it.

The generator refuses a run rather than passing a bad value on to synthesis. Both clock frequencies must be positive, `G_EXTERNAL_TRIG` and `GEN_TYPE` must be 0 or 1, the buffer depth and both FIFO depths must be powers of two greater than one, and the baud rate must be below `G_AXIL_CLK_FREQ/16`. Between `G_AXIL_CLK_FREQ/32` and that limit it runs but prints a warning: the 16× oversampling receiver is down to two clocks per sample there and the margin is thin. The full ranges, including the ones only the elaboration asserts catch, are in the datasheet.

The stock configuration is short enough to read whole:

```
G.SAMP_CLK_FREQ,integer,100000000
G_AXIL_CLK_FREQ,integer,100000000
G.EXTERNAL_TRIG,integer,0
G.SAMP_BUFF_DEPTH,natural,2048
G_UID,natural,0
GEN_TYPE,integer,1
G.UART_RX_FIFO_DEPTH,natural,1024
G.UART_TX_FIFO_DEPTH,natural,1024
G.BAUD_RATE,integer,115200
```

3.3 Writing portmap.csv

Three columns, `signal,width,type`, one signal per line, listed LSB first. The first signal listed owns the low bits of the probe word, and that order is the probe bit order everything else inherits: the `w_probe` concatenation in the core, the trigger vector, the bit numbering both drivers use, and the order the signals appear in the VCD. For the stock AXI4S build that puts `axis_tdata` at bits 0 to 63, `axis_tlast` at 64, `axis_tvalid` at 65 and `axis_tready` at 66, which is exactly what has to be counted out when a trigger pattern is written by hand.

```
# signal,width,type
axis_tdata,64,in
axis_tlast,1,in
axis_tvalid,1,in
axis_tready,1,out
```

Parsing is strict, and deliberately so, since both the generator and the driver have to read this file the same way. Each line is split on commas into exactly three fields, taken literally: a space after a comma lands inside the field, so `axis_tlast, 1, in` is rejected on the type and `axis_tlast ,1,in` would carry the space into the port name. Comments are only comments in column 0, so an indented `#` is an error rather than a comment. Widths have to be positive, names have to be unique, and a portmap with no signals in it is refused. The names are not checked for being legal VHDL identifiers, since that surfaces clearly enough at analysis.

The widths add up to `G_PROBE_WIDTH`, and a sample is stored in $\lceil G_PROBE_WIDTH/32 \rceil$ lanes of 32 bits. A width just over a multiple of 32 therefore costs a whole extra lane for every sample in the buffer: the stock build is 67 bits and spends three lanes on it. It is worth spending the three bits somewhere useful, or trimming a data bus by three, where the choice is free.

`templates/` holds starting points, listed in Table 3.3. Copy one to `codegen/portmap.csv` and cut it down rather than starting from an empty file.

File	Describes
<code>default_portmap.csv</code>	The stock 64-bit AXI4Stream probe, 67 bits in total. This is the build the tests and the shipped drivers assume
<code>axi4s_full_portmap.csv</code>	Every AXI4Stream signal, clock and reset included. Trim the widths to the link
<code>axi4_full_portmap.csv</code>	AXI4 full, all five channels, with the widths taken from Microchip application note AC409
<code>default_configuration.csv</code>	The stock generics, as printed above

Table 3.3: The templates supplied with the generator.

One trap comes from the two tools defaulting differently. For the generator, `--portmap` defaults to the copy under `templates/`; for the python driver it defaults to `codegen/portmap.csv`. The two ship identical, so a stock build works either way, but as soon as one of them is edited the flag has to be passed to both tools. The driver does not have to be trusted to notice: it refuses a readout whose portmap widths do not add up to the probe width the core reports, which is the check that catches a portmap that has drifted away from the synthesised core.

3.3.1 Choosing the signal type

`type` is the direction the signal has on the `probe_slave_` side, which is the side facing the *master* of the probed link. So the portmap carries the directions that link's slave would have, and `probe_master_` gets the mirror of each one. Table 3.4 is the whole of it.

Type	Slave side	Master side	Pass through	Use for
<code>in</code>	input	output	yes, slave to master	Anything the probed link's master drives
<code>out</code>	output	input	yes, master to slave	Anything the probed link's slave drives
<code>mon</code>	input	none	no, parallel tap only	A link that cannot be spliced in series

Table 3.4: The probe signal types.

For AXI4Stream that makes TDATA, TVALID and TLAST `in` and TREADY `out`. A `mon` signal is sampled and can be triggered on like any other, it simply has no master side port and is never driven by the core.

A direction written the wrong way round is not caught anywhere. The generator has no way to know which end of the link drives what, the result elaborates cleanly, and the core still records the signal. What breaks is the pass through, which now drives into the wrong end of the link, so the symptom is a link that stops working the moment the ILA is spliced into it rather than anything the generator or the tools will report.

3.3.2 Using bidirectional signals

There is no `inout` type, and the generator rejects one with an explanation rather than accepting it. A signal assignment cannot pass a bidirectional line through. Writing `probe_master_x <= probe_slave_x` makes the core a second driver on the master side, so anything the far end drives resolves to 'X' and nothing ever propagates backwards, and writing both directions is a combinational loop. Even the open-drain idiom, `'0' when other = '0' else 'Z'` both ways, latches up once both sides are low; real I2C repeaters avoid that with offset voltages, not with logic.

It is not needed either, because an ILA cannot sit between a pad and the outside world. The splice belongs on the fabric side of the IO buffer, where the bus is not bidirectional yet and is still the three unidirectional groups the tristate is built from. For a QPI flash:

```
# signal,width,type
```

```

qspi_cs_n,1,in
qspi_sck,1,in
qspi_sio_o,4,in      # data the controller drives
qspi_sio_oe,1,in     # turnaround control
qspi_sio_i,4,out     # data read back

```

That captures strictly more than a probe on the pad would. Drive value, enable and read value stay separate, so the pad itself is reconstructable, and the turnaround becomes visible and triggerable instead of having to be guessed from a single resolved value, which is where the bugs usually are.

Where the bus genuinely cannot be spliced in series, for instance a controller that only exposes one bidirectional port, tap it in parallel with `mon`. The core reads the resolved value and never drives it.

3.4 Running the generator

```

python3 code-generator.py                    # stock build -> gen_axis/
python3 code-generator.py --portmap portmap.csv \
      --config configuration.csv            # user build  -> gen/
python3 code-generator.py --dry-run         # report only

```

Flag	Effect
<code>--portmap <file></code>	The probe portmap CSV. Default <code>templates/default_portmap.csv</code>
<code>--config <file></code>	The core configuration CSV. Default <code>templates/default_configuration.csv</code>
<code>--outdir <dir></code>	Override the output directory
<code>--dry-run</code>	Report what would be generated, write nothing

Table 3.5: The generator’s flags.

The defaults are resolved relative to the script rather than to the working directory, so it can be run from anywhere, including from the Makefile at the repository root. Paths given on the command line are resolved relative to the working directory, as usual.

Where the output lands is decided by the inputs. A run with *both* CSV files left at their shipped defaults is the stock build and goes to `gen_axis/`; anything else goes to `gen/`. `--outdir` overrides both. Neither directory is tracked, so neither is anything to protect: delete either one and the next run rebuilds it.

The run prints back what it read and what it derived, and that output is worth a glance before synthesis, since the bit ranges are the same ones a hand-written trigger pattern has to be counted against:

```

G.PROBE.WIDTH:67
w_probe(63 downto 0) : axis_tdata
w_probe(64 downto 64) : axis_tlast
w_probe(65 downto 65) : axis_tvalid
w_probe(66 downto 66) : axis_tready

GEN_TYPE 1: generating libre_ila.vhdl, fifo.vhdl, uart.vhdl, libre_ila_uart.vhdl

```

From the repository root, `make configure` brings the stock build up to date and is what the simulations depend on. It is driven by a stamp file in `gen_axis/` that tracks the generator itself, everything in `hdl/` and everything in `codegen/templates/`, so editing a template regenerates on the next run and touching none of them does not run the generator at all.

The trap worth calling out is what that stamp does *not* track. The two CSV files in `codegen/` are not among its dependencies, and neither is a build someone dropped into `gen_axis/` by hand with `--outdir`. Point `--outdir` at `gen_axis/` once and the tests keep simulating that user build until something in `hdl/` or `templates/` changes. Let user builds go to `gen/` where they belong, and if one has already landed in `gen_axis/`, delete the directory or run `make clean`.

3.5 Building a generic logic analyzer

Nothing says the core has to sit in the middle of a link. For a plain logic analyzer, one that only watches a handful of signals scattered around the design, there are two ways to describe it in the portmap: list every signal as `mon`, or list every signal as `in` and leave the master side of the instance open.

Prefer `mon`. There is no master side port to leave dangling, no pass through logic for synthesis to trim away, and the file says what the build is for. With every signal a tap, the `^D0` and `^M0` blocks come out empty and `^SH` becomes a single comment: the core cannot drive anything, by construction rather than by care taken at instantiation.

```
# signal,width,type
dbg_bus,32,mon
dbg_state,4,mon
dbg_valid,1,mon
dbg_error,1,mon
```

The capture side is unchanged by any of this. A `mon` build arms, triggers, fills the same buffer and reads back into the same VCD; the only thing it gives up is the pass through, and a build that observes rather than intercepts had no use for that anyway.

3.6 Probing multiple channels in same build

One core can watch several links at once, as long as they share a sampling clock. List all of them in one portmap and they concatenate into one probe word, with each channel occupying its own run of bits in the order listed. Prefix the names per channel so that the VCD stays readable and the trigger rows in the GUI can be told apart:

```
# signal,width,type
ch0_axis_tdata,32,in
ch0_axis_tvalid,1,in
ch0_axis_tready,1,out
ch1_axis_tdata,32,in
ch1_axis_tvalid,1,in
ch1_axis_tready,1,out
```

This buys something two separate cores cannot do without being chained: there is one trigger vector spanning every channel, so a condition can be written across them, for instance the moment channel 0 asserts `TLAST` while channel 1 is still stalled. It costs width, and the buffer costs its lane count times its depth in RAM, so a wide multi-channel probe is usually paid for by shortening the buffer.

Links on different clocks need separate cores, and two things are worth knowing before wiring that up. Every build emits an entity called `libre_ila`, and `libre_ila_uart` with it, so two *different* probe shapes cannot go into one project without renaming one of the copies. Two instances of the *same* shape are no trouble at all: instantiate the same entity twice and override `G_UID` in each generic map so the host can tell them apart. To make one instance trigger the other, build the second with `G_EXTERNAL_TRIG` set to 1 and drive its `i_ext_trig` from the first one's `o_trig_out`; that pin is not synchronised inside the core, so chained instances have to share a sampling clock or carry a synchroniser between them. The datasheet covers the chaining rules in full.

Chapter 4

Integrating LibreILA into a Design

Chapter 3 ends with a directory of VHDL. This chapter describes how to incorporate it into a design: which files go into the project, which of the two builds to instantiate, where in a link the probe ports go, what to feed the clocks and the reset, and what to look at in the reports once it has been through synthesis. It assumes the user has already generated a core and has a working FPGA project to integrate it into, and has a working knowledge of the FPGA toolchain, especially block design based applications, and of the target device.

None of it is tool specific. The worked example throughout is the supplied project of Chapter 2, a stock `libre_ila_uart` spliced into the AXI4Stream link of a packet generator on the PolarFire SoC Discovery Kit, and the file paths quoted are that project's. Any tool that can compile VHDL-2008 will do the same job.

4.1 Files to add

Once the core has been generated as given in Chapter 3, the HDL files can be added to the project. What the generator emits depends on `GEN_TYPE`, and all of what it emits goes in. Table 4.1 says which file is which.

File	Emitted for	What it is
<code>libre_ila.vhdl</code>	both	The core: sample buffer, trigger, register map and AXI4Lite slave port
<code>libre_ila_uart.vhdl</code>	<code>GEN_TYPE 1</code>	The wrapper: the same core with a UART and a packet parser in front of its bus port
<code>uart.vhdl</code>	<code>GEN_TYPE 1</code>	The serial transmitter and receiver, instantiated by the wrapper
<code>fifo.vhdl</code>	<code>GEN_TYPE 1</code>	The transmit and receive queues, instantiated by the wrapper twice

Table 4.1: The generated files and what each one is.

Only the top one is instantiated by hand. `libre_ila_uart` declares the other three as components and wires them up itself, so a `GEN_TYPE 1` build needs all four files compiled into the project but only one instance written.

Copy them into the project rather than pointing the project at `gen/` where they were generated. That directory is not tracked and the next run of the generator rewrites it, so a project referencing it in place can change under a colleague who has never run the generator at all. In the supplied project the four files sit in `libre_ila_uart_libero/hdl/` alongside the design's own sources.

The sources are plain VHDL-2008 and use only the `ieee` standard libraries: `std_logic_1164`, `numeric_std` and `math_real`. No vendor primitive is instantiated and no vendor library is referenced. The one piece of vendor-specific text anywhere in them is a `syn_ramstyle` attribute of `"lsram"` on the sample buffer, which Synplify and Libero honour and every other tool ignores; removing it changes nothing but the kind of RAM

the buffer lands in. Where a project has a house style, the generated files can be run through VHDL Style Guide first, which is what they were formatted with in the first place.

4.2 Core or wrapper

The choice is about what already exists in the design, not about what the ILA can do, since the capture side is identical either way. `libre_ila` presents the register map on an AXI4Lite slave port and needs a bus master in the design to drive it. `libre_ila_uart` puts that same register map behind a UART, so it needs two fabric pins and a serial adapter instead. Table 4.2 is the comparison.

	<code>libre_ila</code>	<code>libre_ila_uart</code>
Driven by	An AXI4Lite master in the design	A PC, over two wires
Host driver	The C driver, from firmware	The python driver, command line or GUI
Needs	A spare bus master and address space	Two fabric pins and a USB-TTL adapter
Costs at 67 bits by 2048	600 fabric 4LUT, 388 DFF, 7 LSRAM	1280 fabric 4LUT, 817 DFF, 9 LSRAM
Readout of a stock capture	As fast as the master reads it	About 2.9 s at 115 200 baud

Table 4.2: The two builds compared. The resource figures are the datasheet’s, measured on PolarFire SoC.

So the wrapper costs roughly twice the core in fabric and two more LSRAM blocks for its queues, and buys a capture that can be taken from a laptop with nothing running on the chip. Take the core where firmware is already there to drive it and a capture should never need a PC attached, and the wrapper where there is no bus master to spare, or where the debugging is being done from a desk rather than from inside the application.

4.3 Splicing the probe into a link

The two probe ports are the two cut ends of the link. `probe_slave_*` faces the *master* of the probed link and carries the directions that link’s slave would have; `probe_master_*` faces the slave and carries the mirror of each one. So the master’s outputs go into `probe_slave_`, and `probe_master_` goes on to the slave:

master	<code>libre_ila</code>	slave
<code>tvalid</code> $\longrightarrow$	<code>probe_slave_axis_tvalid</code> = <code>probe_master_axis_tvalid</code> $\longrightarrow$	<code>tvalid</code>
<code>tready</code> $\longleftarrow$	<code>probe_slave_axis_tready</code> = <code>probe_master_axis_tready</code> $\longleftarrow$	<code>tready</code>

with the rest of the signals following their own direction the same way. The = in the middle is a signal assignment and nothing more. The splice is combinational and costs the link no latency, no handshake change and no clock cycle; what it does cost is routing, since every probed signal now goes through the core’s ports instead of straight across.

A `mon` signal has no master side port at all and is not a cut. Tap it in parallel with whatever already drives it and leave the link alone.

Where the far end of the link does not exist, the master side ports are the loose ends. The supplied project is exactly that case: the packet generator’s stream ends at the ILA, so `probe_master_axis_tready` is tied to ‘1’ to keep the generator sending and the other `probe_master_` outputs are left unconnected. That is a demonstration rather than a pattern to copy; in a real design both sides go to real logic.

One trap comes with the graphical tools. SmartDesign and its equivalents read the generics off the entity when the component is imported and then write them into the instantiation as explicit overrides, which is why the supplied project’s netlist carries `G_PROBE_WIDTH(67)` in the instance rather than inheriting it. Regenerate with a different portmap and the file on disk changes while the instantiation does not, so the component has

to be refreshed in the tool as well. The mismatch shows up as an elaboration error if you are lucky and as a probe word sliced at the wrong boundaries if you are not.

4.4 Clocks and reset

`samp_aclk` takes the clock the probed link runs on. Not a convenient free-running clock, and not a faster one: the core stores one sample on every rising edge of `samp_aclk`, so anything else records a resampled version of the link rather than the link. `s_axil_aclk` takes whatever the bus runs on, or in a wrapper build, whatever clock the UART should be timed off. The two may perfectly well be the same clock; in the supplied project both are driven from `FIC0_CLK` at 100 MHz.

Both frequencies were fixed at generation, and the generics have to agree with the clocks actually connected here. `G_SAMP_CLK_FREQ` is reported to the host and sets the time axis of the capture, so a mismatch there is a VCD with the wrong time base. `G_AXIL_CLK_FREQ` sets the watchdog period and the UART divider, so a mismatch there is a serial link that does not talk at all. Section 3.2 covers both.

The core takes one reset per domain. `i_rst_sync` is active high, sampled on `samp_aclk`. `s_axil_aresetn` is active low, sampled on `s_axil_aclk`. The wrapper takes only `i_rst_sync` and derives the other from it by inversion, so a wrapper build has a single active-high reset input. In practice, drive it from the reset the rest of the design already uses and hold it for at least two periods of the *slower* of the two clocks. The sample buffer is not cleared by either reset, which matters not at all, because a capture overwrites the whole buffer before it can complete. The full requirements are in the datasheet.

4.5 Timing constraints

`samp_aclk` and `s_axil_aclk` are unrelated clocks. Declare them asynchronous to one another so the tool does not try to close timing between the domains, which in SDC is `set_clock_groups -asynchronous`. Everything that genuinely crosses is already handled inside the core: the arm and disarm toggles and the three status bits carry their own two-flop synchronisers, and the sample buffer crosses inside the RAM, which needs no constraint at all. Where the two clocks come from one source, or the design constrains them together for other reasons, a maximum delay on the crossing registers is the narrower alternative.

This section is work in progress and is the one place in this manual where the advice has not been exercised on hardware. The supplied project runs both domains from a single clock, so no cross-domain constraint was needed to build it and none has been verified in SDC. Treat the paragraph above as the intended constraint rather than a tested recipe, and check the report rather than trusting it.

4.6 Connecting the UART

In a wrapper build, `uart_rx` and `uart_tx` are ordinary one-bit fabric ports. Any two pins that reach the serial adapter will do, and which two is a decision for the design's IO constraints rather than anything in the core. The supplied project puts them on the Discovery Kit's Raspberry Pi header:

```
set_io -port_name uart_tx -pin_name B14 -fixed true \
      -io_std LVCMOS33 -DIRECTION OUT
set_io -port_name uart_rx -pin_name E13 -fixed true \
      -io_std LVCMOS33 -DIRECTION IN
```

Two things about that. The IO standard has to match the adapter, which for the 3.3 V adapters this was tested with is LVCMOS33. And the wires cross: the adapter's TX goes to the pin constrained as `uart_rx`, its RX to the pin constrained as `uart_tx`, and its ground to any ground on the board. Chapter 2 has the wiring for the tested setup.

The line rate is not a runtime setting. `G_BAUD_RATE` was fixed at generation and is baked into the divider, so the host has to be told the same number; changing it means regenerating and resynthesising.

4.7 Checking the build

Two things are worth confirming in the reports before blaming the core for anything: that the sample buffer was inferred as RAM, and that timing is met.

The buffer is the whole cost of the core, and it is meant to land in block RAM. Look at the resource usage by module in the synthesis report and find the core's instance. For the stock build in the supplied project that row reads seven LSRAM (20K) blocks against `libre_ila_inst`, with one more for each of the wrapper's two FIFOs. What a failure looks like is unmistakable: zero RAM blocks against the core, and a flip-flop count several thousand higher than the datasheet's utilization table leads you to expect. That means the tool declined to infer a simple dual-port RAM with independent read and write clocks and fell back to registers. The design still builds and still behaves correctly that way, it just costs far more fabric than it should, and the datasheet's device requirements section covers what to do about it.

For timing, filter the report to the paths that touch the core rather than reading the whole design's worst path. In SmartTime that is a maximum delay analysis with `libre_ila*` in the *From* field, then the same again with it in the *To* field, which separates the paths the core drives from the paths that drive it.

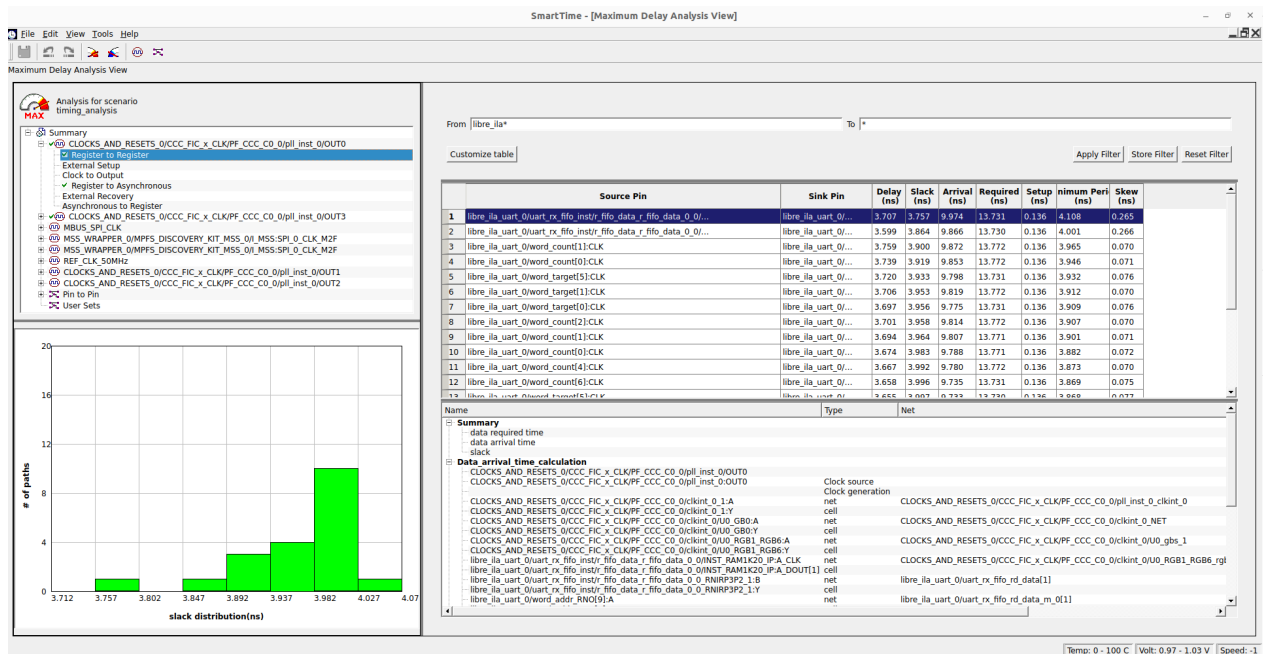


Figure 4.1: Register to register paths out of the core, stock build.

The number to check is simply that the worst slack is positive, on both filters; in Figure 4.1 it is about 3.8 ns. The datasheet quotes f_{max} for several probe widths and depths, and a build that misses timing at a clock well below those figures is usually placement rather than the core: the probe ports fan out across the design by construction, and a core placed far from the link it watches pays for it on the probe paths.

Last, run the design's own tests with the ILA spliced in. A probe direction written the wrong way round in the portmap elaborates cleanly and captures correctly, and shows up only as the probed link no longer working, so a link that misbehaves the moment the core is added is a portmap question and not a timing one. Once the design runs, connecting a host confirms the rest: both drivers read the probe width out of the core and refuse to go on if it disagrees with the portmap they were given, which catches a core synthesised from one portmap and read back with another.

Chapter 5

The Software Stack

This chapter introduces the reader to the python driver stack, what each layer owns, and how they interact. More information on the driver can be obtained by reading the source code and corresponding READMEs.

5.1 Architecture

The driver is five modules under `drivers/python/`, listed in Table 5.1

File	What it owns
<code>libre_ila.py</code>	The command line front end. Every verb lives here
<code>gui.py</code>	The graphical front end, <code>--gui</code>
<code>session.py</code>	One connection to one core, and every operation either front end can ask of it
<code>driver.py</code>	The register map over UART. Knows nothing about what the probe bits mean
<code>vcd.py</code>	The portmap and the VCD. Turns raw samples back into named signals
<code>trigger.py</code>	Per-signal trigger patterns to and from the flat condition and mask pair

Table 5.1: The modules of the python driver.

The `driver.py` serves as the core with `session.py` as the interface to the front ends. The front ends are `libre_ila.py` for the command line and `gui.py` for the GUI. The `session.py` module allows the command line and GUI to share the same operations while leveraging the different lifetimes of a session. The `vcd.py` and `trigger.py` modules are used by the `session.py` to handle the VCD and trigger translation respectively.

5.2 The session

A session is one connection to one core. It is constructed with a serial port, a baud rate and optionally a portmap path and a UID, and it does nothing until `connect()` is called; construction opens no port. Once connected it carries the core's geometry, read out of the core itself rather than configured: `probe_width`, `samp_buff_depth`, `samp_freq_hz`, `n_lanes`, `stride_width` and `uid`. That is the reason re-synthesising with a different probe width needs no change on the host.

The operations are the verbs of Chapter 6 under different names: `info`, `status`, `get_trigger`, `set_trigger`, `set_trigger_position`, `arm`, `force_trigger`, `disarm`, `wait_done` and `capture`. `probes` returns the portmap, loaded on first use and then kept, after checking it against the width the core reports.

Two rules matter when driving one from your own code.

One session may only be touched by one thread at a time. The wrapper is a request and response protocol over a single port, and the transaction layer flushes the input buffer on the way in, so two overlapping operations desync the wrapper and each other. Sessions are independent of one another, one port each, so several cores in parallel is fine; two threads on one session is not.

The readout is the one long operation. `capture()` takes a `progress` callback, called as `progress(words_read, words_total)`, because a full buffer takes seconds rather than a packet. An event driven front end runs it off the thread serving its event loop and reports from there. `wait_done(timeout)` is the blocking alternative, for a front end with nothing else to do, since it polls in a loop; an event driven one polls `status` on a timer instead, which it wants anyway to keep a state display honest.

Failures come back as `OperationError` with a `reason` field taking one of the symbolic values in Table 5.2. The message states the fact and stops, deliberately: the remedy for a capture that has not finished is `--wait-done` on the command line and a button in a window, so the layer that knows which is the one that says so.

Reason	Means
<code>usage</code>	The call itself was wrong, e.g. a trigger value wider than the probe
<code>device-not-found</code>	No stored device with that UID, or its file is unreadable
<code>portmap</code>	No portmap given, or the file could not be read
<code>portmap-mismatch</code>	The portmap widths do not add up to the probe width the core reports
<code>timeout</code>	The link did not answer, or answered out of step
<code>not-done</code>	The capture had not reached DONE when the readout was asked for

Table 5.2: The symbolic reasons an `OperationError` carries.

5.3 The register driver

`driver.py` is the register map over the wire and nothing above it. It reads and writes 32-bit registers by address; it does not know what a probe bit means, what a portmap is, or that a VCD exists.

The wrapper carries AXI4Lite as a request and response packet pair, both fields wider than a byte sent most significant first:

PC to ILA:	0x55	R/W (1)	#words (7)	base address (32)	write words
ILA to PC:	0xAA	valid (1)	#words (7)	base address (32)	read words

So a request is a six byte header and, for a write, four bytes per word; a reply is a six byte header and, for a read, four bytes per word. Both directions are answered, a write included, which is what lets the host tell a lost request from a slow one. The count field is seven bits, so a transfer is at most 127 words and a full buffer readout is split into many packets. The full packet format, including the valid bit and the wrapper's watchdog, is in the datasheet.

The link has two fail safes. Every read carries a timeout, so a wrapper that has stopped answering fails the operation rather than blocking the host forever. And when a reply arrives that does not match its request, the driver resyncs: it reads and discards until the link goes quiet, logging `RESYNC dropped N stale bytes`, and only then reports the failure. Without that, one desynced reply would leave every later transaction reading the tail of the previous one.

5.4 The trigger translator

The core takes a flat condition word and a flat mask word, both as wide as the probe. That is the wrong shape for a person, who thinks in signals: `axis_tvalid` is 1, `axis_tdata` is anything, `axis_tlast` is 1. `trigger.py` translates between the two.

A pattern is written per signal, in one of four radices. `bin` and `hex` carry don't cares, `uint` and `int` do not, since a don't care is a property of a digit and a decimal digit does not map onto a whole number of bits. A don't care is written `x`, `X` or `-`, the last because that is what a truth table uses and someone will type it. Underscores and whitespace are allowed as grouping, so `0b0010_1xxx` is read the way it is written. In hex an `x` is four don't cares; in binary it is one.

Function	Does
<code>parse_pattern</code>	One signal's pattern to a (value, mask) pair of that signal's width
<code>format_pattern</code>	The same pair back to a pattern string, for display
<code>patterns_to_vector</code>	A pattern per probe to the flat condition and mask the core takes
<code>vector_to_patterns</code>	The flat pair back to one pattern per probe

Table 5.3: The trigger translation functions.

The last two take the probe list from the portmap, which is what gives each signal its place in the flat word. The pair round trips: a vector read out of the core becomes patterns, and those patterns become the same vector again, which is what lets a front end show the trigger a core is actually holding rather than the one it last wrote.

5.5 The VCD writer

`vcd.py` owns the portmap and the output file. `load_portmap` reads `portmap.csv` and works out each signal's place in the probe word, returning name, width, type, lsb and msb in the order the file lists them. It applies exactly the acceptance rules of the generator, down to treating an indented `#` as an error rather than a comment, because this file is parsed once to build the hardware and once to read it back and the two have to accept the same set of files.

`check_portmap` is the check that matters. It refuses the readout unless the portmap widths add up to the probe width the core reports. That check exists because the failure it catches is otherwise silent: a portmap that has drifted from the synthesised core still slices the probe word cleanly, just at the wrong boundaries, so the capture comes back looking plausible and meaning nothing.

`unpack_sample` turns one raw row into per-signal values, and `write_vcd` writes the file: signals named from the portmap, the buffer unrolled from the first index, timestamps derived from the sampling clock the core reported, and the trigger carried as an extra one-bit wire. Chapter 9 describes the result from the reader's side.

Chapter 6

The Command Line Interface

`drivers/python/libre_ila.py` drives a `libre_ila_uart` build from a PC. This chapter covers every command it takes, in the order it applies them, and then runs the same capture several ways. It assumes a core already built into a design and reachable over a serial adapter, as in Chapter 4.

Everything the tool writes lands in `capture/` under the working directory: the device store, the `.vcd` and the debug log. The directory is created when something is written, never on import, so a run that only prints leaves nothing behind.

6.1 Telling the tool about a core

A core is introduced once and then addressed by the identity it reports:

```
./libre_ila.py --add-device --serial-port /dev/ttyUSB0 --baud 115200
./libre_ila.py --device 0 --info
```

`--add-device` connects, reads the UID back out of the core and writes `capture/libre_ila_device<UID>.txt`, a single line holding the port and the baud rate. `--reset` forgets every stored device, and `--device-dir` puts the store somewhere else.

The GUI reads the same store, so a core added on the command line comes up as a tab and one added in a tab is reachable with `--device`. That holds only while both are pointed at the same place, which is what `--device-dir` is for: the working directory is the right default for a tool run from a shell and the wrong one for a window started from a desktop entry.

6.2 The verbs and the order they run in

The flags are verbs, and whatever order they are given in, they are applied in the order of Table 6.1. Several of them in one invocation is the normal way to use the tool.

#	Flag	Behaviour
1	<code>--reset</code>	Forget every stored device, then stop
2	<code>--add-device</code>	Store the device on <code>--serial-port</code> , then stop
3		Connect to <code>--device</code>
4	<code>--info</code>	Print what the core reports, then stop
5	<code>--disarm</code>	Cancel a capture in progress, before the trigger writes so a retry is one line
6	<code>--set-trigger-condition</code> , <code>--set-trigger-mask</code> , <code>--set-trigger-type</code> , <code>--set-trigger-reduction</code>	Merged into one trigger write
7	<code>--set-trigger-position</code>	Where the trigger sits in the buffer, in samples
8	<code>--get-trigger-configuration</code>	Read the trigger back out of the core
9	<code>--arm</code>	
10	<code>--force-trigger</code>	
11	<code>--wait-done [SECONDS]</code>	Default 10
12	<code>--read-data</code>	Write the <code>.vcd</code> to <code>--output</code>

Table 6.1: The verbs, and the fixed order they are applied in.

So a whole capture is one line:

```
./libre_ila.py --set-trigger-condition 0x2000000000000000 \
               --set-trigger-mask 0x2000000000000000 \
               --set-trigger-type rising --arm --wait-done --read-data -o cap.vcd
```

and splitting it across invocations works just as well: arm now, read whenever. Nothing about the session is kept on the host. The core holds the trigger setup, the state and the samples, and the driver reads its whole geometry back out of the core at startup, which is the same reason re-synthesising with a different probe width needs no change here.

6.3 Configuring the trigger

`--set-trigger-condition` is the pattern the probe word is compared against, and `--set-trigger-mask` picks which of its bits are compared. Both take one number covering the whole probe word, in any base: `0x...`, `0b...`, `0o...` or decimal.

These two flags **do not use the portmap names**. Bit 0 is the first signal listed in `portmap.csv` and the bits run upwards from there, so the number has to be counted out. For the stock AXI4S build `axis_tvalid` is bit 65, so triggering on it rising is:

```
--set-trigger-condition 0x2000000000000000 \
--set-trigger-mask      0x2000000000000000 \
--set-trigger-type      rising
```

`--set-trigger-type` is `level`, `rising` or `falling`, and `--set-trigger-reduction` is `and`, every masked bit matches, or `or`, any one of them. The two are independent. Whichever fields are left out keep whatever the core already had, because the trigger registers read back before they are written, so setting one field does not disturb the rest.

A value with bits above the probe width is refused rather than trimmed. This is worth the strictness: the core zero-extends each sample to the full register stride before comparing, so a mask bit up in the padding fires an `or` trigger on the very first sample and stops an `and` trigger from firing at all. Both failures look like the trigger, not like the number.

6.4 Worked scenarios

Trigger on a condition. Arm on the rising edge of TVALID and read the result back in one line:

```
./libre_ila.py --device 0 \
  --set-trigger-condition 0x2000000000000000 \
  --set-trigger-mask 0x2000000000000000 \
  --set-trigger-type rising --set-trigger-position 512 \
  --arm --wait-done 30 --read-data -o packet_start.vcd
```

Force a trigger that never came. When the condition turns out not to occur, arm and then force, which fills the buffer from wherever it had got to:

```
./libre_ila.py --device 0 --arm
./libre_ila.py --device 0 --force-trigger --wait-done --read-data
```

Disarm and retry. `--disarm` runs before the trigger writes, so changing your mind about the condition is one line rather than two invocations:

```
./libre_ila.py --device 0 --disarm \
  --set-trigger-type level --set-trigger-reduction or --arm
```

A capture split across two invocations. Arm before firing whatever the design does, then come back for the data:

```
./libre_ila.py --device 0 --arm
# ... run the design
./libre_ila.py --device 0 --wait-done 0 --read-data -o late.vcd
```

The last one is the case that shows where the state lives. The core stays armed and then DONE across the gap, and the second invocation reads its geometry, its trigger and its samples back out of the core with nothing carried over from the first.

6.5 Debugging the link

`--debug` logs every UART transaction to `capture/libre_ila.log`. It works with every verb and with `--gui`, where all tabs share the one file.

```
2026-08-13 12:49:30 /dev/ttyACM0 connect baudrate=115200
2026-08-13 12:49:30 /dev/ttyACM0 TX read addr=0x00000004 count=5 bytes=55 05 00 00 00 04
2026-08-13 12:49:30 /dev/ttyACM0 RX header bytes=aa 85 00 00 00 04
2026-08-13 12:49:30 /dev/ttyACM0 RX payload words=0xb01dface 0x05f5e100 0x00000043 0
x00000800 0x00000000
```

Every line carries the port, so two tabs writing at once stay apart, and the file is appended to and flushed per line, so a run that hangs or is killed still leaves everything up to that point on disk.

ERR lines carry the failure, and a **RESYNC dropped N stale bytes** line follows one when the driver drained the link to get back in step. **The byte counts are worth reading, not just the errors.** The wrapper answers a read with a six byte header and four bytes per word, so a reply that is short or long for the request it followed says the two ends disagree about which request is being served, rather than that one reply got corrupted. The extract above is a healthy exchange to compare against: a five word read of the output block, answered with a header whose count matches and five payload words, the first of which is the magic key.

6.6 Exit statuses

Code	Cause
0	Success
2	Usage error: the flags themselves do not make sense, or a trigger value is wider than the probe
3	Could not add the device: the port did not open, or nothing answered on it
4	No such stored device, or its file is unreadable. Usually <code>--device</code> given an index instead of a UID
5	No portmap: <code>--portmap</code> points at nothing, or the default is not there
6	The portmap does not match the core: its widths do not add up to the probe width the core reports
7	Link error: no port, busy port, timeout or a desynced reply
8	The capture did not reach DONE within <code>--wait-done</code>
9	The core refused the operation, e.g. arming an armed ILA or disarming an idle one
10	The GUI is not available, i.e. PySide6 is not installed

Table 6.2: Exit statuses of the command line front end.

Codes 6, 7, 8 and 9 are the four that mean something is wrong rather than mistyped, and each has an entry in Chapter [10](#) with the fix.

Chapter 7

The Graphical Interface

The graphical front end drives the same `libre_ila_uart` build as Chapter 6, through the same session layer, and does the same things: connect, set a trigger, arm, watch, capture, open the result. What it adds is that the trigger is edited per signal, with the portmap's names in front of you, rather than counted out into one long number.

7.1 Starting it

```
./libre_ila.py --gui
```

It needs PySide6, which is the only dependency in this project that the command line does not share; Section 2.1.1 covers installing it, including on distributions that do not package it. Without it the tool exits with status 10 rather than failing at import, so the rest of the driver keeps working on a machine that has no Qt at all.

`--waveform-viewer` and `--portmap` mean the same as they do on the command line and are passed straight through. So does `--device-dir`, and it matters more here: the window reads the same device store as `--device`, so a core added in one place shows up in the other only while both are pointed at the same directory. The working directory is the right default for a tool run from a shell and the wrong one for a window started from a desktop entry, which is the case `--device-dir` exists for.

7.2 Tabs and connections

One tab per stored device, titled `CONNECTED ILA<UID>` or `DISCONNECTED ILA<UID>`, so the two things worth knowing at a glance are in the tab label. `+ ADD TAB` introduces a core the same way `--add-device` does, and it lands in the same store.

A tab owns one session for as long as it lives, so the port is opened once and held rather than reopened per button press. That is the property the whole design rests on: the wrapper is a request and response protocol on a single port, so a connect, several status polls, a trigger read and an arm all have to go through one open port in one order. It is also why the status poll timer stops for the whole readout, since an overlapping poll would desync the wrapper.

A tab starts disconnected. Opening the window opens no ports, which means a window left open does not hold a serial port that something else may want, and a board that is not plugged in yet is not an error at startup. Press **Connect** when the board is there.

7.3 Running a capture

The screenshots through this section are the two fake cores of `tests/python/05_gui`, rendered with `make shots`, so every step below can be followed with no hardware attached.

1. **Connect.** A tab opens with its controls idle and its geometry unknown, since nothing has been read yet. Pick the port and baud rate on the right and press **Connect**. The ILA information box fills in with what the core reports: UID, probe width, buffer depth and sampling frequency. Those four are read out of the core, so they are also the quickest check that you are talking to the core you meant to.

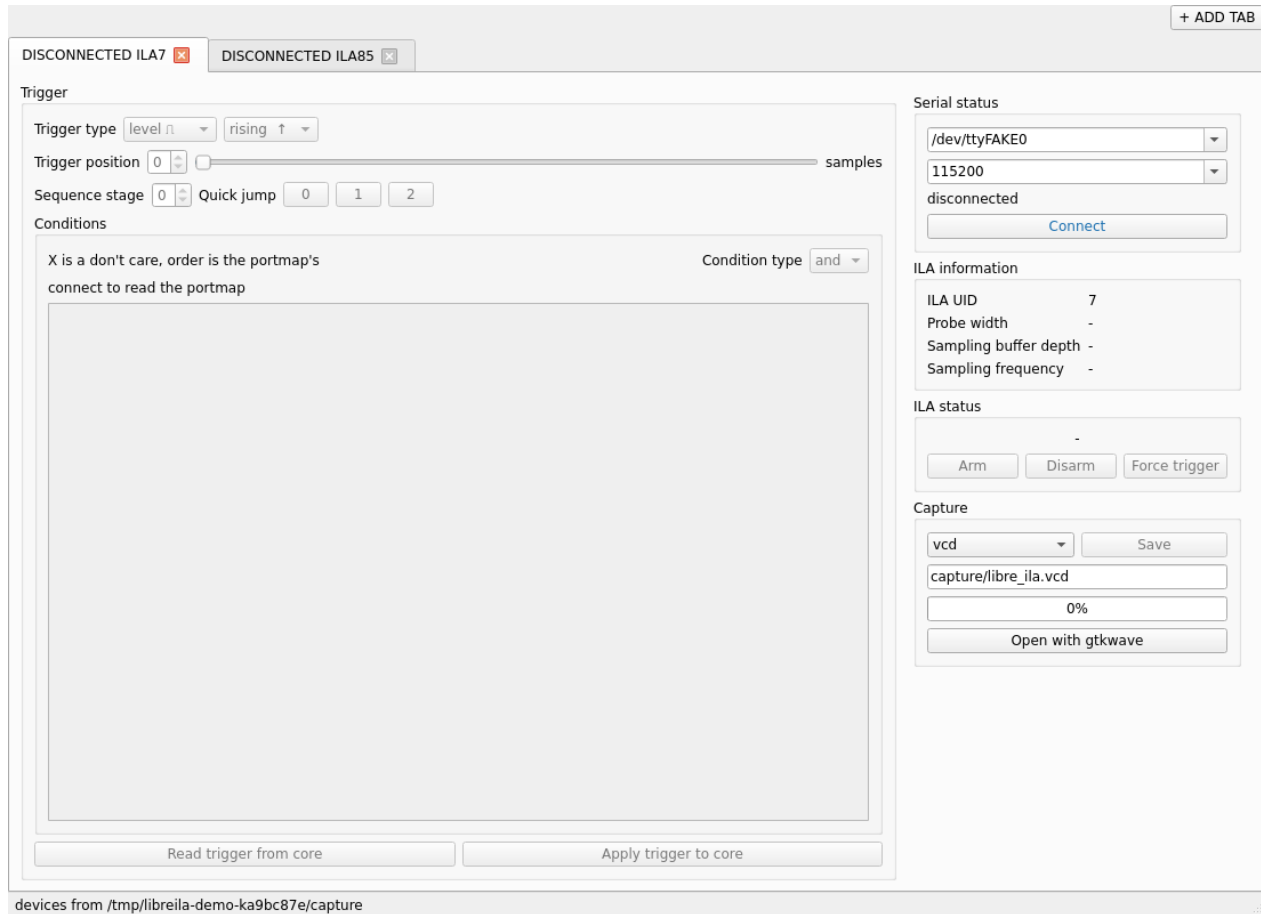


Figure 7.1: Two stored cores, both disconnected. The geometry reads - because only the UID is known from the device store.

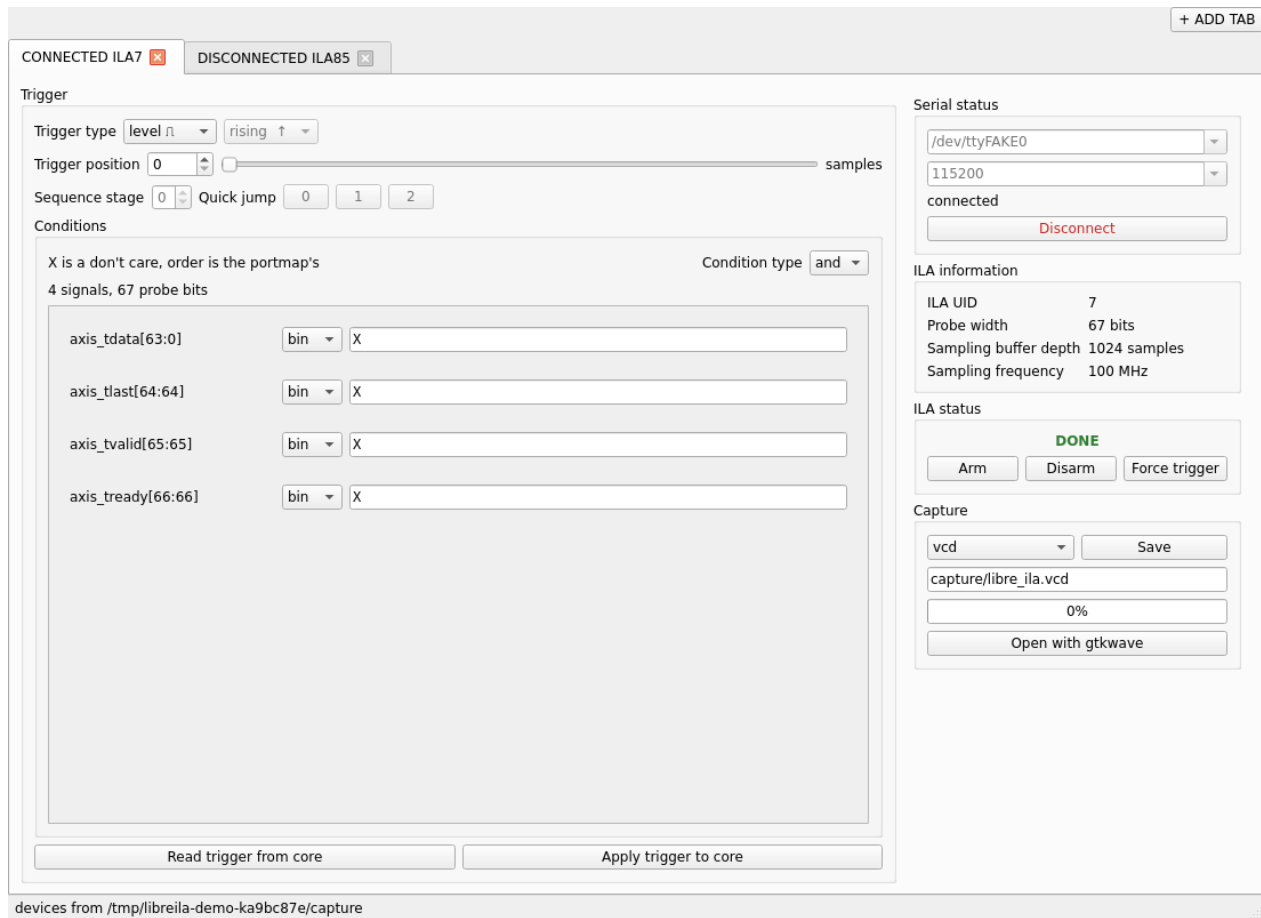


Figure 7.2: The first tab connected. The information box and the conditions table both come from the core.

2. **Set the trigger.** Read `trigger` from `core` loads what the core is currently holding. The conditions table has one row per signal in portmap order, each with a radix and a pattern, where `X` is a don't care; **Trigger type** chooses level or edge and, for edge, rising or falling; **Condition type** chooses whether every signal has to match or any one of them. **Trigger position** is where in the buffer the trigger sits, in samples.

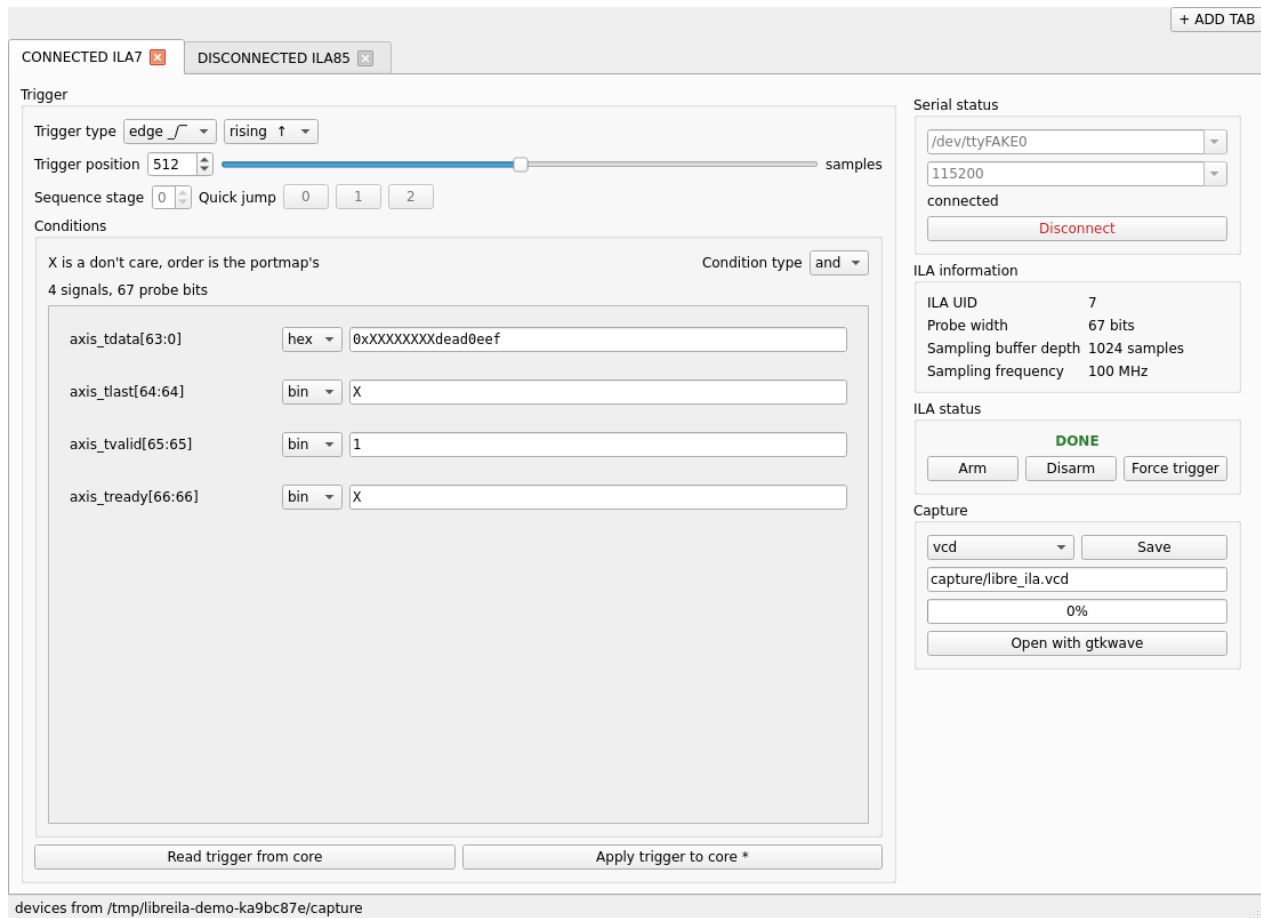


Figure 7.3: An edited trigger: TDATA matched in hex with the top eight nibbles left as don't cares, TVALID high, the rest ignored, on a rising edge with 512 samples of history. The * on the apply button says the core has not been told yet.

3. **Apply it.** Apply trigger to core writes the table down as the flat condition and mask pair. Until it is pressed the button carries a *, so an edited table that has not been written is visible rather than assumed.

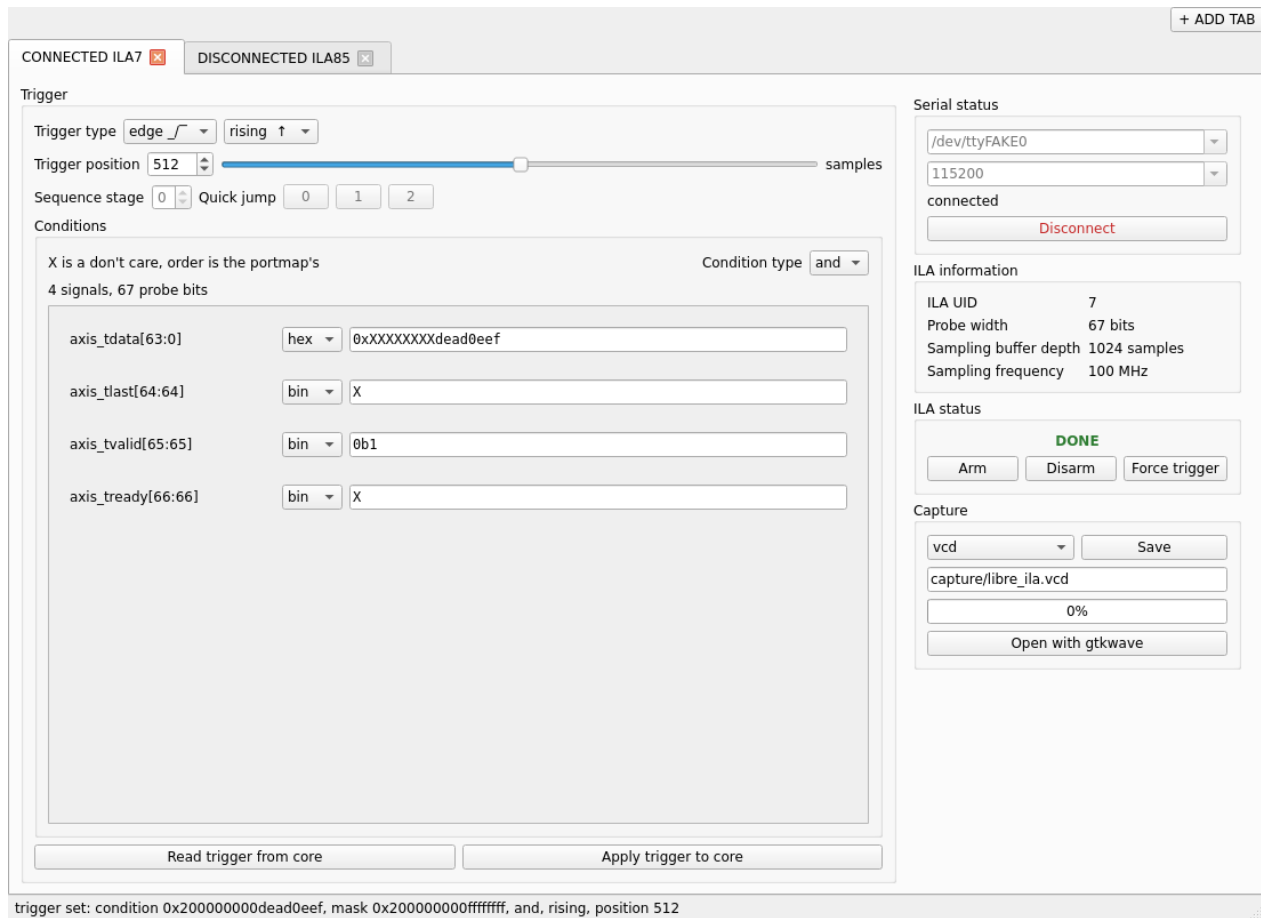


Figure 7.4: Applied. The status bar shows the flat pair the table became, which is the same pair `--set-trigger-condition` and `--set-trigger-mask` take, and the rows now read back as the core holds them.

4. **Arm.** The status box goes to **ARMED** with a running timer, so a trigger that never comes looks like a trigger that never comes rather than like a hung window.

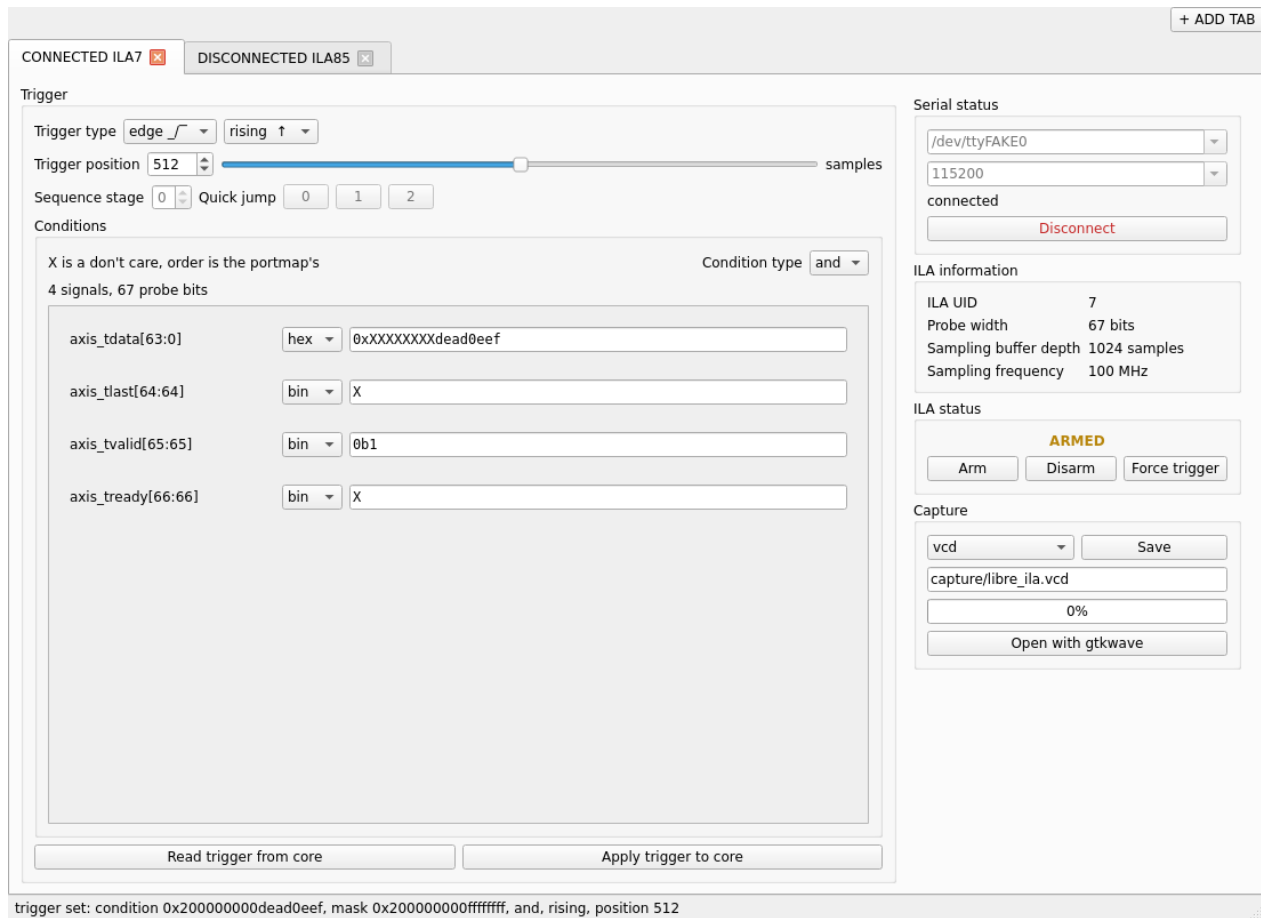


Figure 7.5: Armed and waiting, with the timer counting.

5. **Force or disarm if needed.** `Force trigger` fills the buffer from wherever the capture had got to; `Disarm` returns the core to idle.
6. **Capture.** Once the status reads `DONE`, `Save` reads the buffer out to the path in the box below it, with the progress bar moving as it comes in. The readout runs off the thread serving the window, which is why the window stays responsive for the seconds it takes.

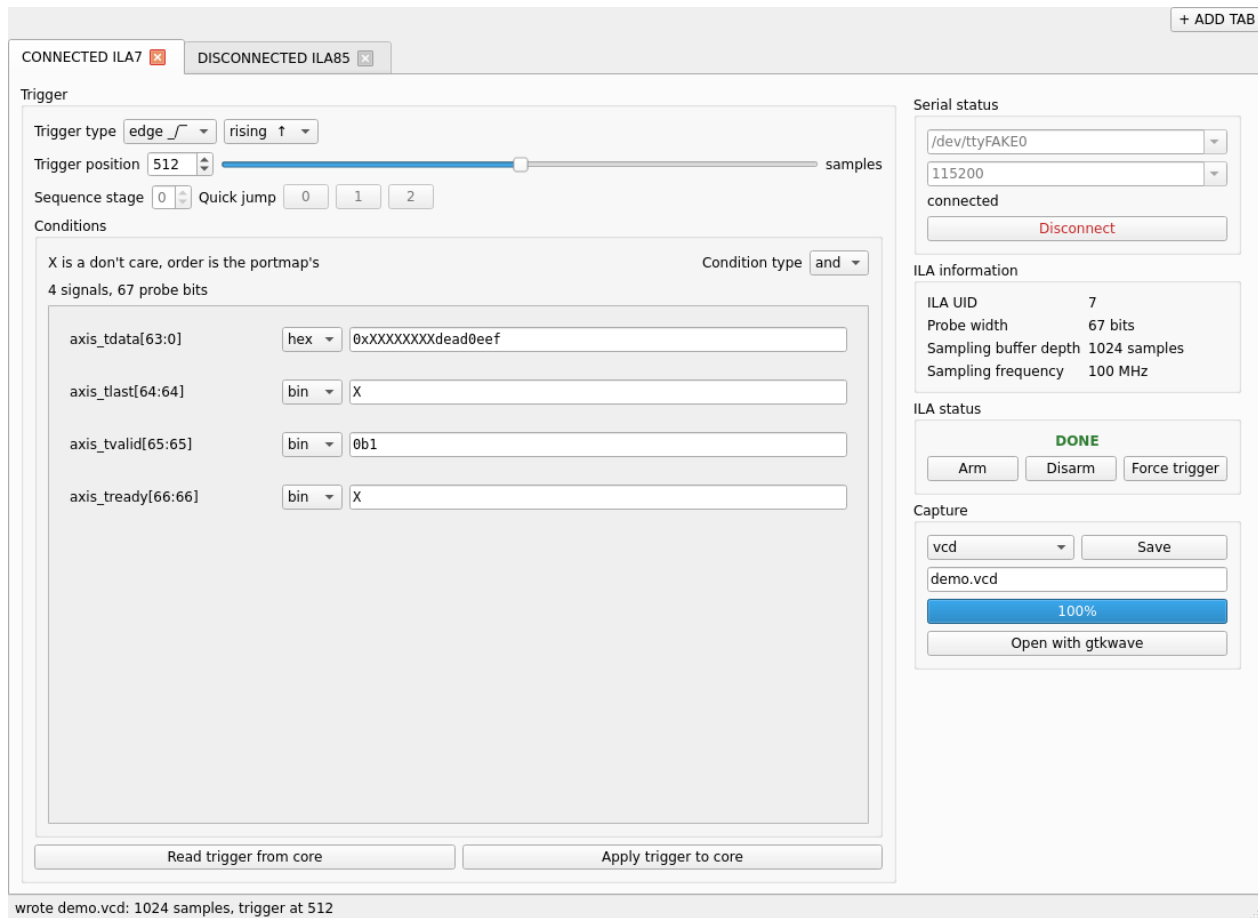


Figure 7.6: Read out and written. The status bar gives the sample count and the row the trigger fired on.

7. **Open it.** Open with `gtkwave` launches the waveform viewer on the file just written. Chapter 9 covers what is in it.

Figure 7.4 is worth a second look, because it is the whole reason to prefer this front end for setting a trigger. The table above the status bar and the two long numbers in it are the same trigger: the window does the counting that Section 6.3 has to do by hand.

A refusal from the core, such as disarming an ILA that is already idle, reaches the status bar rather than the event loop. The window survives it and says what happened; nothing about a refused operation closes the connection or loses the tab.

7.4 What it does not do yet

The `Sequence stage` and `Quick jump` controls are visible but disabled, and their tooltip says why: the sequencer needs a core with sequencer registers and this one has none. `TRIG.CFG` defines bits 2:0 and nothing else, so there is no register for a stage to live in. They are shown rather than hidden so that the shape of the trigger panel does not change when a core that has them arrives.

The capture format list offers `vcd` and nothing else so far, which is the same one format the command line writes.

Everything else the command line can do, this can do. If a control you want is missing, the corresponding verb in Chapter 6 still exists, and both front ends can be pointed at the same core in turn, since neither keeps any state of its own between operations.

Chapter 8

The Baremetal C Driver

This chapter is the whole of the `libre_ila` path. Where the previous two chapters drive the UART wrapper from a PC, this one drives the bare core over its AXI4Lite slave port from firmware running on the same chip, so a capture never leaves the device and no host has to be attached to take one. A reader on this path needs nothing from the python driver and this chapter assumes nothing from those chapters.

What it does assume is a core already built into a design and mapped into the address space of a processor that can reach it, as in Chapter 4, with `GEN.TYPE` set to 0.

8.1 Adding the driver to a project

Three files under `drivers/baremetal/`, listed in Table 8.1, all of which go into the firmware project.

File	What it is
<code>core_libre_ila.c</code>	The implementation
<code>core_libre_ila.h</code>	The API: the instance type, the status and mode enums, and the function prototypes
<code>core_libre_ila regs.h</code>	Register offsets, field masks, and the macros that place a probe bit or index a sample

Table 8.1: The files of the baremetal driver.

The register accesses go through the HAL macros `HAL_get_32bit_reg` and `HAL_set_32bit_reg` and the two `HW*_reg` functions under them, which is the one place the driver is tied to a toolchain. **As shipped it assumes the Microchip PolarFire SoC toolchain**, whose HAL supplies those macros, `addr_t` and `readmtime()`. On that toolchain the driver compiles as it stands. On any other, Section 8.7 is the checklist of what to reimplement; it is short.

The header includes `hal/hal.h`, or `hal.h` when `LEGACY_DIR_STRUCTURE` is defined, which is the difference between SoftConsole project layouts rather than anything about the ILA.

8.2 Definitions you must supply

Three defines, and they size the arrays you declare and nothing else. The driver reads none of them.

Define	Is
CORE_LIBRE_ILA_PROBE_WIDTH	Total probed bits, G_PROBE_WIDTH in the HDL
CORE_LIBRE_ILA_SAMP_BUFF_DEPTH	Depth of the sample buffer in samples, G_SAMP_BUFF_DEPTH in the HDL
CORE_LIBRE_ILA_SAMP_FREQ_HZ	Sampling clock in Hz, G_SAMP_CLK_FREQ in the HDL. Nothing is derived from this one at all; it is somewhere to record what you expect, and the core reports the real value into <code>ila.samp_clk_freq_hz</code>

Table 8.2: The build time definitions, which size arrays and nothing else.

Array sizes cannot come from the hardware. They are storage in your own scope and there is no allocator on the way, so the size stays a build time decision however much the rest of the driver refuses to be. What the driver does instead is take the length alongside the pointer: `LIBRE_ILA_configure_trigger()` and `LIBRE_ILA_read_data()` both return `CMD_STATUS_BAD_PARAM` unless the length matches what the core actually needs, and write nothing.

That is worth the extra argument. A width guessed wrong fails at the call rather than overrunning the buffer, and the match is exact rather than "at least" on purpose: the sample macros index rows at the lane count they were compiled with, so a buffer sized for a wider probe than the core has would be filled at one row length and read back at another.

8.3 Initialising

```
libre_ila_instance_t ila;

if (LIBRE_ILA_init(&ila, MY_ILA_BASE_ADDR) != CMD_STATUS_SUCCESS) {
    /* no LibreILA at that address, or it reports a geometry it
       could not have been elaborated with */
}
```

`LIBRE_ILA_init()` first reads `MGCKEY` and checks it against `0xB01DFACE`, which is what turns a wrong base address into `CMD_STATUS_BAD_MAGIC_KEY` rather than into nonsense. It then reads the probe width, the buffer depth and the sampling clock frequency out of the core and works the four block base addresses out from them, leaving the lot in the instance.

Nothing in the driver is compiled against one synthesis. Re-synthesising with a different `G_PROBE_WIDTH` or buffer depth needs no firmware rebuild, only arrays large enough. This works because the output block sits at the base address in every build, so the registers that report the width are findable before the width is known; the datasheet's register summary lays that out.

`init()` also picks up `ila.uid`, the core's `G_UID`. It is carried rather than checked, since every value is legal and zero simply means the core was built without one. Where the magic key answers "is this a LibreILA", the UID answers "which one".

8.4 Configuring the trigger

The probe is opaque to the driver. It sees one flat word of `CORE_LIBRE_ILA_PROBE_WIDTH` bits, bit 0 upwards, and both the trigger vector and the captured samples use that same layout. Naming the bits is your project's job, and the pattern is a define per bit plus the two placement macros:

```
#define MY_PROBE_ERR_BIT (3u)

uint32_t cond[LIBRE_ILA_TRIG_WORDS(CORE_LIBRE_ILA_PROBE_WIDTH)] = {0};
uint32_t mask[LIBRE_ILA_TRIG_WORDS(CORE_LIBRE_ILA_PROBE_WIDTH)] = {0};

cond[LIBRE_ILA_BIT_WORD(MY_PROBE_ERR_BIT)] |= LIBRE_ILA_BIT_MASK(MY_PROBE_ERR_BIT);
mask[LIBRE_ILA_BIT_WORD(MY_PROBE_ERR_BIT)] |= LIBRE_ILA_BIT_MASK(MY_PROBE_ERR_BIT);
```

```
LIBRE_ILA_configure_trigger(&ila, cond, mask, ila.stride_width,
                           LIBRE_ILA_TRIG_MODE_AND | LIBRE_ILA_TRIG_EDGE);
```

`cond` is the value each probe bit has to take and `mask` picks which bits take part, a zero leaving that bit a don't care. Words above the probe width are padding and must stay zero, for the same reason the python driver refuses a value with bits up there: the core zero-extends each sample to the full stride before comparing.

The mode argument carries two independent things ORed together. `LIBRE_ILA_TRIG_MODE_AND` or `LIBRE_ILA_TRIG_MODE_OR` is the reduction over the enabled bits, and `LIBRE_ILA_TRIG_EDGE`, optionally with `LIBRE_ILA_TRIG_FALLING`, decides whether the trigger follows the level of that reduction or one of its edges. Level is the default and fires on the first sample if the condition already holds when the ILA is armed, which is rarely what a request like "trigger when the error bit is high" actually means.

8.5 Running a capture

The whole sequence, as one function that compiles:

```
#include "core_libre_ila.h"

#define MY_ILA_BASE_ADDR (0x60000000u)
#define MY_PROBE_ERR_BIT (3u)

#define TRIG_WORDS LIBRE_ILA_TRIG_WORDS(CORE_LIBRE_ILA_PROBE_WIDTH)
#define BUFF_WORDS LIBRE_ILA_SAMPLE_WORDS(CORE_LIBRE_ILA_PROBE_WIDTH, \
                                           CORE_LIBRE_ILA_SAMP_BUFF_DEPTH)

static uint32_t samp_buffer[BUFF_WORDS];

int capture_on_error(void)
{
    libre_ila_instance_t ila;
    uint32_t cond[TRIG_WORDS] = {0};
    uint32_t mask[TRIG_WORDS] = {0};
    uint32_t trig_row = 0u;
    uint32_t n;

    if (LIBRE_ILA_init(&ila, MY_ILA_BASE_ADDR) != CMD_STATUS_SUCCESS) {
        return -1;
    }

    /* keep a quarter of the buffer as history before the trigger */
    LIBRE_ILA_set_trigger_position(&ila, ila.samp_buff.depth / 4u);

    cond[LIBRE_ILA_BIT_WORD(MY_PROBE_ERR_BIT)] |= LIBRE_ILA_BIT_MASK(MY_PROBE_ERR_BIT);
    mask[LIBRE_ILA_BIT_WORD(MY_PROBE_ERR_BIT)] |= LIBRE_ILA_BIT_MASK(MY_PROBE_ERR_BIT);

    if (LIBRE_ILA_configure_trigger(&ila, cond, mask, ila.stride_width,
                                   LIBRE_ILA_TRIG_MODE_AND | LIBRE_ILA_TRIG_EDGE)
        != CMD_STATUS_SUCCESS) {
        return -1;
    }

    LIBRE_ILA_arm(&ila);

    /* ... let the design run ... */

    if (LIBRE_ILA_wait_done(&ila, 5000u) != CMD_STATUS_SUCCESS) {
        /* the error never happened: take what there is anyway */
        LIBRE_ILA_force_trigger(&ila);

        if (LIBRE_ILA_wait_done(&ila, 1000u) != CMD_STATUS_SUCCESS) {
            LIBRE_ILA_disarm(&ila);
            return -1;
        }
    }
}
```

```

    if (LIBRE_ILA_read_data(&ila, samp_buffer, BUFF_WORDS, &trig_row)
        != CMD_STATUS_SUCCESS) {
        return -1;
    }

    /* samp_buffer now holds samp_buff_depth samples, oldest first, and
       sample trig_row is the one the ILA triggered on */
    for (n = 0u; n < ila.samp_buff_depth; n++) {
        if (LIBRE_ILA_SAMPLE_BIT(samp_buffer, n, MY_PROBE_ERR_BIT)) {
            /* ... */
        }
    }

    return 0;
}

```

Three things in there are worth pointing out. `LIBRE_ILA_arm()` ignores the value written, since the hardware acts on the write itself. `LIBRE_ILA_wait_done()` polls until DONE or the timeout, so its failure is a trigger that has not fired rather than a broken core, which is why forcing is the sensible next step. And `read_data()` returns the trigger position *already rebased* on its own output: the buffer comes back unrolled, oldest sample first, and `trig_row` indexes it directly. `LIBRE_ILA_read_idx()` is there for a caller who wants the raw circular indices instead.

Samples are stored packed, `n_lanes` words each, with the padding words the hardware inserts up to the register stride dropped. So word `w` of sample `n` is `samp_buffer[n * n_lanes + w]`, which is what `LIBRE_ILA_SAMPLE_WORD()` spells, and probe bit `b` of sample `n` is `LIBRE_ILA_SAMPLE_BIT()`.

8.6 Several cores from one binary

Each instance carries its own geometry, so one binary can drive cores of different probe widths at once. Give each its own `libre_ila_instance_t`, initialise each from its own base address, and size its arrays with the width macros:

```

libre_ila_instance_t ila_a, ila_b;
uint32_t buff_a[LIBRE_ILA_SAMPLE_WORDS(96u, 4096u)];
uint32_t buff_b[LIBRE_ILA_SAMPLE_WORDS(35u, 1024u)];

LIBRE_ILA_init(&ila_a, ILA_A_BASE);
LIBRE_ILA_init(&ila_b, ILA_B_BASE);

/* index the wider one at its own lane count, not the global default */
value = LIBRE_ILA_SAMPLE_BIT_N(buff_a, ila_a.n_lanes, sample, bit);

```

The `_N` variants of the sample macros take the lane count explicitly, which is what a second core of a different width needs; the plain `LIBRE_ILA_SAMPLE_WORD()` and `LIBRE_ILA_SAMPLE_BIT()` use the global `CORE_LIBRE_ILA_PROBE_WIDTH` and are only right for a core that matches it.

The magic key and the UID divide the work of finding them. The key says a given base address holds a LibreILA at all; the UID says which one, so firmware scanning a set of addresses does not have to hardcode which peripheral sits where. Set `G_UID` per core in `configuration.csv` and they stay distinguishable for the life of the build.

8.7 Porting to another toolchain

The register accessors are the only tool-dependent part of the driver. Everything else is plain C99. To move it to a toolchain that is not Microchip's, supply the following and change nothing in `core_libre_ila.c`:

- `addr_t`, an integer type wide enough to hold a peripheral address.
- `uint32_t HW_get_32bit_reg(addr_t addr)` and `void HW_set_32bit_reg(addr_t addr, uint32_t value)`, a 32-bit volatile read and write at an absolute address. On a Xilinx toolchain these are `Xil_In32` and `Xil_Out32` under different names.

- `HW_get_32bit_reg_field()` and `HW_set_32bit_reg_field()`, the same access with a shift and a mask applied.
- The four `HAL_*_reg` macros that wrap them. These paste `REG_NAME##_REG_OFFSET` onto a base address at preprocess time, which is the contract the register map is laid out around: offsets in `core_libre_ila_regs.h` are relative to their block, and the part that moves with the probe width lives in the block base instead, so an offset worked out at runtime never has to go through them.
- `readmtime()`, a free-running monotonic counter, used only by `LIBRE_ILA_wait_done()` for its timeout. Its tick rate is `LIBRE_ILA_MTIME_HZ`, which defaults to 1 MHz and can be defined to whatever the counter actually runs at. A port that does not need a timeout can poll `LIBRE_ILA_get_status()` itself and skip this one.

A worked example of exactly that substitution is in the repository: `tests/c/00_reg_map/stub/` backs those accessors with a flat array so the whole driver compiles and runs on a host, which is what lets the C driver be tested with no hardware and no vendor toolchain present. [Appendix A](#) runs it.

Chapter 9

Reading the Capture

A finished capture is one file, and this chapter is about reading it: what is in it, where the trigger is, what the time axis means, and how to unroll the buffer yourself if you are on the baremetal path and have raw words rather than a file.

9.1 The VCD file

`--read-data` writes a VCD, which every waveform viewer reads. The signals are named from `portmap.csv`, in the order that file lists them, and they sit in a scope called `libre_ila`, so a capture opened next to a simulation of the same design does not collide with it. The header is short enough to read by eye:

```
$timescale 1ps $end
$scope module libre_ila $end
$var wire 64 ! axis_tdata $end
$var wire 1 " axis_tlast $end
$var wire 1 # axis_tvalid $end
$var wire 1 $ axis_tready $end
$var wire 1 % trigger $end
$upscope $end
$enddefinitions $end
```

The body is delta encoded. The first sample states every signal, and after that only what moved, which is what keeps a full buffer of a mostly idle bus small. It also means a signal that never changes appears once, at time zero, and nowhere else.

The one thing that can go wrong before the file is written is the portmap. The driver refuses the readout unless the widths in `portmap.csv` add up to the probe width the core reports, and exits with status 6 if they do not. That check is there because the failure it catches is otherwise silent: a portmap that has drifted from the synthesised core still slices the probe word cleanly, just at the wrong boundaries, so the capture comes back looking entirely plausible and meaning nothing. A name that has moved is easy to spot; a 64-bit bus read off one bit low is not.

Keep the portmap a build actually used with that build, in other words. If a core has been re-synthesised from a different portmap since, pass `--portmap` pointing at the right one rather than trusting the default.

9.2 The trigger marker

VCD has no notion of a marker, cursor or annotation, so the trigger is carried as data: an extra one-bit wire named `trigger`, declared after the probe signals, which is 0 for every sample except the one the ILA triggered on, where it is 1. It pulses for exactly that sample.

That sample is the one the condition held on, in level mode, or the one it changed on, in the edge modes. There is no pipeline delay to correct for on the host side, so the pulse marks the event itself rather than the core's reaction to it.

Declaring it last is deliberate: adding or removing a signal in the portmap does not shift the identifier the trigger wire uses, so scripts that look for it keep working across builds.

9.3 The time axis

Timestamps are in picoseconds, one sample apart, at the sampling clock the core reports. Picoseconds keep the step an exact integer for every clock that divides into 10^{12} , which covers the round numbers a sampling domain actually runs at; 100 MHz lands on 10 000 ps per sample.

The rate comes from `G_SAMP_CLK_FREQ` as synthesised, read back out of the core rather than assumed by the host. So a generic that does not match the clock actually connected to `samp_aclk` gives a perfectly correct capture on a wrong time axis: every sample is real and in the right order, and every interval measured off the viewer's cursor is wrong by the ratio of the two frequencies. Nothing in the flow can detect this, since the core has no way to measure its own clock. It is worth checking the frequency in `--info` against the design once per build.

9.4 In the waveform viewer

The GUI's `Open with gtkwave` launches the viewer on the file just written, and on the command line the file is opened however you would open any other VCD. `--waveform-viewer` names a different one for a reader who does not use GTKWave; anything that takes a file path on its command line will do.

In GTKWave, the signals are not shown until they are added: right click `libre_ila` in the tree, then `Recurse Import` and `Insert`, which brings in the probe signals and the trigger wire in portmap order. The magnifying glass with the green tick fits the whole capture in the window, which is the quickest way to see the shape of what was caught. Figure 2.12 in Chapter 2 is a real capture opened this way.

To find the trigger, look for the pulse on `trigger`, or search on that signal going high. Everything to the left of it is history the core was already holding when the condition fired, and its extent is what `--set-trigger-position` chose; everything to the right was captured after.

9.5 Reading the raw buffer yourself

On the baremetal path, or from a custom host, what comes back is words rather than a file. The buffer is circular and the capture stopped wherever it stopped, so the words are not in time order until they are unrolled.

Two registers say where the window is, and both are valid once `DONE` is set. `SAMP_BUFF_FRST_IDX` is the oldest sample, `SAMP_BUFF_TRIG_IDX` is the one that triggered, and both are raw indices into the circular buffer. Then, with `DEPTH` the buffer depth:

- Sample i of the window, counting from the oldest, is at buffer index $(\text{FRST_IDX} + i) \bmod \text{DEPTH}$.
- The trigger sits at position $(\text{TRIG_IDX} - \text{FRST_IDX}) \bmod \text{DEPTH}$ within the unrolled window.

Skipping the unroll is a recognisable failure rather than a subtle one: the capture reads correctly from the start index to the end of the buffer, then jumps backwards in time to the part that was overwritten, so the waveform looks right up to a discontinuity and wrong after it.

Within a sample, the probe word is stored one 32-bit lane per slice, lane n carrying probe bits $32n + 31$ down to $32n$, with the last lane zero padded in its upper bits. On the bus each sample is padded further, up to the register stride, so a 67-bit probe reads back as four registers per sample of which the first three carry data. `LIBRE_ILA.read_data()` does all of this for you, including the unroll and dropping the padding words, and hands back the trigger position already rebased on its own output; Section 8.5 is that call in context. For a host doing it by hand, the register offsets and the full capture procedure are in the datasheet.

Chapter 10

Troubleshooting

Every entry here starts from a symptom you can actually see, rather than from a part of the system. Table 10.1 is the index; the sections after it carry the detail.

Symptom	Section
The simulations are testing a build I did not generate	10.1
Invalid type for probe or Invalid portmap line	10.1
directives never found aborts the generator	10.1
The core costs thousands of flip-flops and no RAM blocks	10.1
Timing fails on paths between the two clock domains	10.1
Elaboration complains about the probe width after regenerating	10.1
No such port, or permission denied on /dev/ttyUSB0	10.2
The port is busy	10.2
Exit status 7, or a timeout with nothing in the log	10.2
RESYNC dropped N stale bytes	10.2
The ILA triggers the instant it is armed	10.3
The ILA never triggers	10.3
Exit status 6, the portmap does not match the core	10.4
The signals are named wrongly, or shifted by a few bits	10.4
The waveform jumps backwards in time part way through	10.4
Every interval measured in the viewer is wrong by the same ratio	10.4
The capture came from the wrong ILA	10.4

Table 10.1: Symptom to section.

10.1 Generation and build

The simulations are testing a build I did not generate. The testbenches read `codegen/gen.axis/`, and the stamp that keeps it up to date tracks the generator, `hdl/*.vhd1` and `codegen/templates/*.csv`. It does not track `codegen/portmap.csv`, `codegen/configuration.csv`, or a build someone wrote into `gen.axis/` by hand with `--outdir`. Point `--outdir` there once and the tests keep simulating that user build until something in `hdl/` or `templates/` changes. Delete the directory, or run `make clean` at the repository root, and let user builds go to `gen/` where they belong.

Invalid type for probe or Invalid portmap line. The portmap parser takes each line as exactly three comma separated fields, literally, so a space after a comma lands inside the field: `axis_tlast, 1, in`

is rejected on the type, and `axis_tlast ,1,in` would carry a space into the port name. A `#` that is not in column 0 is an error rather than a comment, deliberately, since the same file is parsed once to build the hardware and once to read it back. Section 3.3 has the full rules.

directives never found aborts the generator. A file listed as carrying a `^^XX` directive no longer has it, which normally means a template in `hdl/` was edited and a comment header went with the edit. Put the directive line back. The generator treats this as an error rather than as nothing to do because a silently dropped block does not fail at generation at all; it fails much later, at elaboration, as an unconnected port.

The core costs thousands of flip-flops and no RAM blocks. The synthesis tool declined to infer a simple dual-port RAM with independent read and write clocks for the sample buffer and fell back to registers. The design still builds and still behaves correctly, it just costs far more fabric than it should. Check the resource usage by module: the core's instance should carry a non-zero RAM block count, as in Section 4.7. The datasheet's device requirements section covers the inference itself.

Timing fails on paths between the two clock domains. `samp_aclk` and `s_axil_aclk` are unrelated, and if the constraint file never says so the tool will try to close timing between them. Declare them asynchronous, as in Section 4.5. Everything that genuinely crosses already carries its own synchroniser inside the core.

Elaboration complains about the probe width after regenerating. Graphical tools capture the generics into the instantiation when the component is imported, so the regenerated file on disk and the stale override in the block design disagree. Refresh the component in the tool. This is the good outcome; the bad one is an override that still elaborates and slices the probe word at the wrong boundaries, which shows up later as a capture whose signals are all shifted.

10.2 The link

No such port, or permission denied. The adapter is not enumerated, or your user is not in the `dialout` group. Add yourself once and log out and back in, as in Section 2.1.1. A permission error names the port, so it is easy to tell from the port not existing at all.

The port is busy. Something else holds it: a GUI tab that is still connected, a serial terminal left open, or a previous run that has not exited. A tab holds its port for as long as it is connected, on purpose, so disconnect the tab rather than killing the window.

Exit status 7, or a timeout with nothing in the log. The request went out and nothing came back. In order of how often it is the cause:

- **Baud mismatch.** `G.BAUD_RATE` is fixed at synthesis and `--baud` has to match it. Neither end can detect the difference, so the symptom is silence rather than corruption.
- **RX and TX not crossed.** The adapter's TX goes to the pin constrained as `uart_rx` and its RX to the pin constrained as `uart_tx`. Straight through wiring gives exactly this symptom.
- **The wrong port.** Several devices enumerate at once on a board like the Discovery Kit, and only one of them is the ILA's adapter.
- **The FPGA is not programmed,** or is held in reset, so nothing is listening.

Run with `--debug` to tell them apart: a TX line with no RX line after it is one of the four above, whereas an RX line that arrives and is rejected is the next entry.

RESYNC dropped N stale bytes. A reply arrived that did not match its request, so the driver drained the link to get back in step before reporting the failure. Read the byte counts around it, not just the error: the wrapper answers a read with a six byte header and four bytes per word, so a reply that is short or long for the request it followed says the two ends disagree about *which* request is being served, rather than that one reply was corrupted. The usual cause is two things talking to one core at once, which is also why a session may only be touched by one thread at a time.

10.3 The trigger

The ILA triggers the instant it is armed. This is the single most common surprise, and it is almost always a level trigger whose condition already held at the moment of arming. In level mode the core fires on

the very first sample if the condition is already true, so "trigger when TVALID is high" on a busy stream captures the moment you pressed the button and nothing of interest. Use an edge mode, which waits for a transition and does not mistake an already true condition for a rising edge.

The other cause is an all-zero mask in AND mode, which matches *vacuously*: with no bits enabled, every enabled bit trivially matches, so the condition is true from the first sample. If the intent is never to trigger on the probe, use an all-zero mask in OR mode instead, which is the combination that genuinely never fires.

The ILA never triggers. Work through the trigger from the outside in:

- **The mask.** An all-zero mask in OR mode never fires, as above. A mask that enables the wrong bits fires on the wrong thing or on nothing.
- **The reduction.** AND asks for every enabled bit at once, which for two signals that are never simultaneously in the required state can be unsatisfiable. OR asks for any one of them.
- **Bits above the probe width.** The core zero-extends each sample to the full register stride before comparing, so a mask bit set up in the padding fires an OR trigger on the first sample and stops an AND trigger from ever firing. The command line refuses these values rather than trimming them; a host writing the registers directly has to keep the padding words zero itself.
- **The condition simply has not happened.** Use `--force-trigger`, or the `Force trigger` button, which fills the buffer from wherever the capture had got to. What comes back is still real data, just not centred on anything.

Reading the trigger back with `--get-trigger-configuration` is worth doing before changing anything, since it shows what the core is actually holding rather than what you meant to write.

10.4 The capture

Exit status 6, the portmap does not match the core. The widths in `portmap.csv` do not add up to the probe width the core reports. Either `--portmap` is pointing at the wrong file, or the core was re-synthesised from a different portmap since. Note that the generator and the driver default to *different* files: `templates/default_portmap.csv` and `codegen/portmap.csv` respectively. They ship identical, so edit one and pass `--portmap` to both.

The signals are named wrongly, or shifted by a few bits. The portmap has drifted from the synthesised core in a way the width check cannot see: reordering two signals, or moving a bit between two of them, leaves the total width unchanged. Regenerate the core from the portmap you are reading it with, or read it with the portmap it was generated from.

The waveform jumps backwards in time part way through. The circular buffer was read without unrolling from `SAMP_BUFF_FRST_IDX`, so the capture reads correctly from the start index to the end of the buffer and then continues with the part that was overwritten. Section 9.5 has the arithmetic. Both supplied drivers do this for you, so this one belongs to a custom host.

Every interval measured in the viewer is wrong by the same ratio. `G_SAMP_CLK_FREQ` does not match the clock actually connected to `samp_aclk`. The capture itself is correct and in order; only the time axis is wrong, because the core has no way to measure its own clock and reports what it was told at synthesis. Check the frequency in `--info` against the design, and regenerate with the right value.

The capture came from the wrong ILA. `--device` selects by the UID the core reports, not by the order devices were added, and it defaults to 0. A core built without a `G_UID` reports 0, so several of them in one system are indistinguishable and the tool will simply talk to whichever one is stored under that name. Give each core its own `G_UID` in `configuration.csv` and rebuild.

Appendix A

Simulations

Everything in this manual except programming a board can be exercised with nothing plugged in. The test suite is how, and it is also the quickest way to find out whether a change you made broke something, since the host side of it runs in milliseconds.

A.1 The test suite

From the root of the repository:

```
make sim           # everything
make sim-python    # the host side tests, python3 and nothing else
make sim-c         # the baremetal driver, a C compiler and nothing else
make sim-hdl       # the GHDL simulations
make configure     # just bring the generated core up to date
```

`tests/` is split by what a test *needs* to run rather than by what it covers, which is what makes the three targets independently useful. Table A.1 is the whole dependency story.

Group	Needs	Covers
<code>tests/python/</code>	python3	The python driver, every layer of it, against a byte level model of the wrapper
<code>tests/c/</code>	A C compiler	The baremetal driver, compiled and run on the host against a stubbed HAL
<code>tests/hdl/</code>	GHDL, and the generated core	The core itself, simulated as the generator actually emits it

Table A.1: The three test groups and what each one needs.

`make sim` runs the host side first, and deliberately: those tests take milliseconds, so a broken packet format or register map fails there rather than after the simulations have run. A directory counts as a test if it carries its own `Makefile`, which keeps the numbering a convention rather than something the build depends on.

Two notes on what is not needed. The python group needs no serial port and no board, because it drives the driver against a model of the wrapper rather than against hardware; `tests/python/05_gui` additionally needs PySide6 and skips itself where the widget packages are absent, so `make sim-python` really does need python3 and nothing else. The C group needs no vendor toolchain, because `tests/c/00_reg_map/stub/` supplies what the PolarFire SoC HAL would have; that stub is also the worked example of the port described in Section 8.7.

A.2 What each test proves

The HDL simulations are the specification of the core's behaviour, one directory per property.

Directory	Proves
00_sim_ext_trig	The external trigger build triggers off <code>i_ext_trig</code>
01_sim_axil_trig	The internal trigger fires on a condition, here <code>TLAST</code> high
02_sim_axil_trig_data	A masked data pattern triggers on the data it names and not on the rest
03_sim_axil_cdc	The two clock domains behave when they are not the same clock
04_sim_axil_force_trig	A second write to <code>ARM_FT</code> forces the trigger
06_sim_libre_ila_uart	The UART wrapper end to end: configure, arm, trigger and read back over the serial link
07_sim_edge_test	Level against edge triggering, <code>TRIG_CFG</code> bits 1 and 2
08_sim_addr_bound_check	Address decoding at and past the end of the register map
09_sim_axil_disarm	The <code>DISARM</code> register returns an armed core to <code>IDLE</code>

Table A.2: The HDL simulations.

The host side tests cover the two drivers layer by layer.

Directory	Proves
python/00_pkt_format	The register reads and writes against a byte level model of the wrapper's packet parser
python/01_vcd	The portmap parsing that turns <code>portmap.csv</code> into a bit layout, and the VCD that comes out of it
python/02_cli	The command line front end end to end, every verb, against that same model
python/03_session	The session layer both front ends drive
python/04_trigger	The translation between per-signal patterns and the flat condition and mask pair
python/05_gui	The graphical front end, driven offscreen against two fake cores
c/00_reg_map	The baremetal driver against the register map, rather than against itself

Table A.3: The host side tests.

The HDL group reads `codegen/gen_axis/` rather than `hdl/`, so what is simulated is what the generator actually emits rather than the templates it emits from. Each test Makefile regenerates that directory when the generator, anything in `hdl/` or anything in `codegen/templates/` has changed under it, so running one test directly stays as up to date as going through the root Makefile. Section 3.4 covers the one case that stamp cannot see.

A.3 Running one on its own

Every test directory carries its own Makefile and can be run from where it is:

```
cd tests/hdl/07_sim_edge_test
make sim      # run it
make wave     # run it and open the result in gtkwave
make clean
```

make wave is the useful one when a simulation fails: the waveform it opens is the core's own internals, not a capture, so it shows what the trigger comparator and the capture state machine were doing at the moment the test gave up.

The graphical front end has one more entry point, which is not a test at all:

```
cd tests/python/05_gui
make gui      # open the real GUI against two fake cores
make shots    # the same, rendered offscreen to shots/*.png
```

make gui starts the real window against a throwaway device store holding two fake cores, so the whole of Chapter 7 can be followed without a board attached. It is the fastest way to see what the tool does before committing to building a core into a design.